Virgo と Aurora で学ぶ ゼロ知識証明入門

透明性・ポスト量子安全性・低次数テスト・GKR/IOP の観点から

本 PDF 版は、Zenn Book 原稿として作成された章立てをもとに、読書向けの目次・組版・ページ構成を加えて再編集したものです。

透明なゼロ知識証明系である Aurora と Virgo を、初学者から学部生でも追えるように、概念・数式・ユースケース・実装上の注意まで含めて体系的に解説します。

PDF Edition

2026 年 4 月 6 日

目次

まえがき **1**
この本の狙い . . . 1
本書が重視すること . . . 1
読者に求める前提 . . . 2
本書の読み方 . . . 2
まず全体像を掴みたい人 . . . 2
数式も含めて腰を据えて学びたい人 . . . 2
記号と約束 . . . 2
先に一言でいうと . . . 3
文献上の事実と本書の解釈 . . . 3
本書で使う主要文献 . . . 3
ここから先の心構え . . . 4

1 ゼロ知識証明の基礎 **5**
1.1 ゼロ知識証明とは何か . . . 5
1.2 3 つの基本性質 . . . 5
1.2.1 完全性（completeness） . . . 5
1.2.2 健全性（soundness） . . . 5
1.2.3 ゼロ知識性（zero knowledge） . . . 6
1.3 直感的な例：年齢証明 . . . 6
1.4 証明したいのは「知識」か「計算」か . . . 6
1.5 関係 R と NP 的な見方 . . . 6
1.6 計算はなぜ回路にするのか . . . 7
1.7 小さな具体例 . . . 7
1.8 なぜ「短い証明」が欲しいのか . . . 8
1.9 SNARK, SNARG, STARK という言葉 . . . 8
1.9.1 SNARK / SNARG . . . 8
1.9.2 STARK . . . 8
1.10 trusted setup と transparent setup . . . 8
1.11 でも「transparent なら常に勝ち」ではない . . . 9
1.12 この本での設計観 . . . 9
1.13 この章のまとめ . . . 9

2 Aurora / Virgo を理解するための下準備 **11**
2.1 算術回路とは何か . . . 11
2.2 layered arithmetic circuit . . . 11
2.2.1 なぜ layer 化がうれしいのか . . . 12
2.3 R1CS とは何か . . . 12
2.3.1 例：$z = xy + x + 2$ を R1CS にする . . . 12
2.3.2 なぜ R1CS が便利なのか . . . 13
2.4 なぜ多項式が出てくるのか . . . 13
2.4.1 低次数性の直感 . . . 13
2.5 Reed–Solomon 符号 . . . 14
2.6 関数の拡張：multilinear extension . . . 14
2.7 IOP とは何か . . . 15
2.7.1 なぜ IOP がうれしいのか . . . 15
2.7.2 非対話化はどうするのか . . . 15
2.8 sumcheck の直感 . . . 15
2.8.1 1 ラウンド目 . . . 16
2.8.2 2 ラウンド目 . . . 16

2.8.3 最後 . . . 16
2.8.4 なぜこれが効くのか . . . 16
2.9 FRI の直感 . . . 16
2.9.1 何をしたいのか . . . 17
2.9.2 直感的な折りたたみ . . . 17
2.10 GKR の直感 . . . 17
2.10.1 layer ごとの値表 . . . 17
2.10.2 wiring predicate . . . 17
2.11 「structured circuit」とは何か . . . 18
2.12 Aurora と Virgo の違いを先取りする . . . 18
2.12.1 Aurora 的世界観 . . . 18
2.12.2 Virgo 的世界観 . . . 18
2.13 この章のまとめ . . . 18

3 Aurora の全体像 20
3.1 Aurora とは何か . . . 20
3.2 Aurora が解きたい問題 . . . 20
3.3 Aurora の基本アイデア . . . 21
3.4 なぜ R1CS から多項式へ行くのか . . . 21
3.5 Aurora を構成する主要部品 . . . 21
3.5.1 1. R1CS . . . 21
3.5.2 2. Encoded IOP . . . 21
3.5.3 3. Univariate sumcheck の新しい IOP . . . 21
3.5.4 4. FRI . . . 22
3.5.5 5. Merkle コミットメントと Fiat–Shamir . . . 22
3.6 Aurora の複雑性をどう読むか . . . 22
3.6.1 proof size が短いとはどういうことか . . . 22
3.6.2 verifier が $O(n)$ なのは弱点なのか . . . 22
3.7 Aurora の何が新しかったのか . . . 23
3.8 Aurora の直感的な図式 . . . 23
3.9 Aurora が向く場面 . . . 23
3.10 Aurora と STARK の関係をどう見るか . . . 24
3.11 Aurora を読むときの心の置き場 . . . 24
3.12 この章のまとめ . . . 24

4 Aurora を手で追う 25
4.1 例題 . . . 25
4.2 Step 1: R1CS に落とす . . . 25
4.2.1 ここでの本質 . . . 25
4.3 Step 2: 証明者が大きな表を作る . . . 26
4.4 Step 3: 制約違反を多項式恒等式へまとめる . . . 26
4.4.1 どうして一括主張が嬉しいのか . . . 26
4.5 Step 4: univariate sumcheck 的な圧縮 . . . 27
4.5.1 なぜ和を見るのか . . . 27
4.6 Step 5: でも“都合のよい表”を出されたら困る . . . 27
4.7 Step 6: そこで FRI を使う . . . 27
4.7.1 重要な直感 . . . 28
4.8 Step 7: オラクルは全部見せない。Merkle で固定する . . . 28
4.9 Step 8: Fiat–Shamir で非対話化する . . . 28
4.10 Step 9: ゼロ知識性はどう確保するのか . . . 28
4.11 1 本の Aurora 証明に何が入っているか . . . 29
4.12 例題に戻ると何が起きているのか . . . 29
4.13 Aurora の理解でつまずきやすい点 . . . 29
4.13.11. 「FRI だけで全部証明している」と思う . . . 29
4.13.22. 「R1CS をそのままランダム点で見ている」と思う . . . 30
4.13.33. 「proof size が短いなら verifier も極小だろう」と思う . . . 30
4.13.44. 「ゼロ知識性はハッシュしているから自動で得られる」と思う . . . 30
4.14 Aurora を一行で再定義する . . . 30

4.15 この章のまとめ 30

5 Virgo の全体像 **31**
5.1 Virgo とは何か 31
5.2 Virgo が解きたい問題 31
5.3 Virgo の backbone は GKR 31
5.4 でも GKR だけでは Virgo にならない 32
5.5 zkVPD とは何か 32
5.5.1 なぜ VPD が必要なのか 32
5.6 Virgo の大まかな流れ 33
5.7 structured circuit とは何か 33
5.7.1 直感的意味 33
5.7.2 どんな回路が structured か 34
5.8 Virgo の複雑性をどう理解するか 34
5.8.1 何が嬉しいのか 34
5.8.2 verifier の見方 34
5.9 Virgo の concrete result 34
5.10 Virgo の新しさを一言でいうと 35
5.11 Virgo が向く場面 35
5.12 Virgo のトレードオフ 35
5.12.11. layered / structured への依存 35
5.12.22. 実装と理論の距離 35
5.12.33. 後続研究による再検討 35
5.13 Aurora と見比べたときの第一印象 36
5.14 Virgo を読むときの心の置き場 36
5.15 この章のまとめ 36

6 Virgo を手で追う **37**
6.1 例題となる layered circuit 37
6.2 Step 1: 各 layer の値を表にする 37
6.2.1 入力 layer 37
6.2.2 中間 layer 37
6.2.3 出力 layer 38
6.3 Step 2: その表を multilinear extension に持ち上げる 38
6.4 Step 3: 検証したい主張を 1 個に圧縮する 38
6.5 Step 4: GKR が layer 0 の主張を layer 1 へ落とす 38
6.5.1 何を sumcheck するのか 39
6.6 Step 5: sumcheck による圧縮のイメージ 39
6.7 Step 6: 同じことを layer 1 から入力 layer へ繰り返す 39
6.7.1 これのどこがすごいのか 40
6.8 Step 7: では $\tilde{V}_i(r)$ は誰がどう計算するのか 40
6.9 Step 8: zkVPD の役割を直感で理解する 40
6.9.1 GKR と zkVPD の役割分担 40
6.10 Step 9: structured circuit が効く場所 41
6.11 Step 10: 非対話化とゼロ知識化 41
6.12 この例で verifier が本当に納得するもの 41
6.13 Virgo の理解でつまずきやすい点 41
6.13.11. GKR が“表の全部を見ている”と思う 41
6.13.22. zkVPD が GKR を置き換えていると思う 42
6.13.33. verifier が常に速いと思う 42
6.13.44. Virgo は単に“FRI の代わりに別の PCS を使っただけ”と思う 42
6.14 Virgo を一行で再定義する 42
6.15 この章のまとめ 42

7 Aurora と Virgo の比較 **43**
7.1 まずは早見表 43
7.2 比較 1: 入力表現の違い 43
7.2.1 Aurora は R1CS が自然言語 43
7.2.2 Virgo は layered circuit が自然言語 44

7.2.3 どちらが“より一般”か . . . 44
7.3 比較 2: 多項式を見る幾何学の違い . . . 44
7.3.1 Aurora は univariate / evaluation domain の幾何学 . . . 44
7.3.2 Virgo は multilinear / hypercube の幾何学 . . . 44
7.3.3 教訓 . . . 45
7.4 比較 3: 何を圧縮しているのか . . . 45
7.4.1 Aurora が圧縮しているもの . . . 45
7.4.2 Virgo が圧縮しているもの . . . 45
7.5 比較 4: verifier の性格 . . . 45
7.5.1 Aurora verifier . . . 45
7.5.2 Virgo verifier . . . 46
7.5.3 実務的な読み方 . . . 46
7.6 比較 5: prover の性格 . . . 46
7.6.1 Aurora prover . . . 46
7.6.2 Virgo prover . . . 46
7.6.3 どう解釈するか . . . 46
7.7 比較 6: 実装難度の違い . . . 46
7.7.1 Aurora の実装難所 . . . 46
7.7.2 Virgo の実装難所 . . . 47
7.7.3 どちらが難しいか . . . 47
7.8 比較 7: どんな回路に向くか . . . 47
7.8.1 Aurora が向きやすい回路 . . . 47
7.8.2 Virgo が向きやすい回路 . . . 47
7.8.3 重要な補足 . . . 47
7.9 Aurora と Virgo は競合か、補完か . . . 47
7.10 どちらを選ぶべきか：簡易意思決定表 . . . 48
7.10.1 Aurora を第一候補にしやすいとき . . . 48
7.10.2 Virgo を第一候補にしやすいとき . . . 48
7.10.3 どちらでもなく、他系も見るべきとき . . . 48
7.11 研究者にとっての学習価値 . . . 48
7.12 この章のまとめ . . . 48

8 ユースケースと選び方 50
8.1 注意：この章の「ユースケース」は設計上の適合例である . . . 50
8.2 ユースケース 1: Merkle 木の証明 . . . 50
8.2.1 問題設定 . . . 50
8.2.2 なぜ Virgo と相性がよいのか . . . 50
8.2.3 Aurora ではどうか . . . 51
8.2.4 まとめ . . . 51
8.3 ユースケース 2: バッチ検証・ロールアップ的計算 . . . 51
8.3.1 問題設定 . . . 51
8.3.2 Aurora が自然に見える状況 . . . 51
8.3.3 Virgo が自然に見える状況 . . . 51
8.3.4 まとめ . . . 52
8.4 ユースケース 3: プライバシー保護された資格証明・KYC . . . 52
8.4.1 証明したい計算 . . . 52
8.4.2 Aurora 的に見ると . . . 52
8.4.3 Virgo 的に見ると . . . 52
8.4.4 設計メモ . . . 52
8.5 ユースケース 4: 行列演算・機械学習・線形代数 . . . 53
8.5.1 例 . . . 53
8.5.2 なぜ Virgo 的か . . . 53
8.5.3 ただし注意 . . . 53
8.6 ユースケース 5: クラウドへの計算委託（verifiable outsourcing） . . . 53
8.6.1 Aurora 的に自然な見方 . . . 53
8.6.2 Virgo 的に自然な見方 . . . 54
8.7 ユースケース 6: 再利用される固定回路 . . . 54
8.7.1 何を見るべきか . . . 54

8.8 逆に、どんなときに相性が悪いか . . . 54
8.8.1 Aurora が相対的に苦しく見えやすいとき . . . 54
8.8.2 Virgo が相対的に苦しく見えやすいとき . . . 54
8.9 実践的な選び方のフローチャート . . . 54
8.9.1 質問 1: 入力は自然に R1CS になるか、layered circuit になるか . . . 55
8.9.2 質問 2: verifier の軽さはどれくらい重要か . . . 55
8.9.3 質問 3: 回路はどれだけ規則的か . . . 55
8.9.4 質問 4: 実装上の制約は何か . . . 55
8.10 この章のまとめ . . . 55

9 実装・研究・セキュリティ上の注意点 56
9.1 まず最初に：論文理解と本番採用は別物 . . . 56
9.2 文献上の事実 1: Virgo は後続研究で再検討されている . . . 56
9.2.1 どう読むべきか . . . 56
9.3 文献上の事実 2: FRI の Fiat–Shamir 安全性も後続研究で精密化された . . . 57
9.4 文献上の事実 3: “lightweight cryptography”は“実装が簡単”を意味しない . . . 57
9.5 設計上の助言 1: まず“回路の自然な形”を見よ . . . 57
9.5.1 Aurora 寄りのサイン . . . 58
9.5.2 Virgo 寄りのサイン . . . 58
9.6 設計上の助言 2: フィールド選択は“最後の微調整”ではない . . . 58
9.7 設計上の助言 3: range check と bit decomposition を甘く見ない . . . 58
9.8 設計上の助言 4: メモリを計算時間と同じくらい重視する . . . 59
9.8.1 Aurora 側で起きやすいこと . . . 59
9.8.2 Virgo 側で起きやすいこと . . . 59
9.9 設計上の助言 5: transcript の domain separation を徹底する . . . 59
9.10 設計上の助言 6: “structured”を曖昧語のまま使わない . . . 59
9.11 設計上の助言 7: Aurora でも Virgo でも、ゼロ知識の確認は別枠で行う . . . 60
9.12 実験を書くときの注意 . . . 60
9.13 研究テーマとして何が面白いか . . . 60
9.14 実務的なまとめ . . . 61
9.15 この章のまとめ . . . 61

10 演習問題 62
10.1 基礎確認 . . . 62
10.1.1 問題 1 . . . 62
10.1.2 問題 2 . . . 62
10.2 回路と R1CS . . . 62
10.2.1 問題 3 . . . 62
10.2.2 問題 4 . . . 63
10.2.3 問題 5 . . . 63
10.3 多項式と低次数性 . . . 63
10.3.1 問題 6 . . . 63
10.3.2 問題 7 . . . 63
10.4 sumcheck と GKR . . . 64
10.4.1 問題 8 . . . 64
10.4.2 問題 9 . . . 64
10.4.3 問題 10 . . . 64
10.5 Aurora / Virgo の理解確認 . . . 64
10.5.1 問題 11 . . . 64
10.5.2 問題 12 . . . 64
10.5.3 問題 13 . . . 65
10.6 実装と研究の視点 . . . 65
10.6.1 問題 14 . . . 65
10.6.2 問題 15 . . . 65
10.6.3 問題 16 . . . 65
10.7 発展問題 . . . 66
10.7.1 問題 17 . . . 66
10.7.2 問題 18 . . . 66

10.8学習の目安 66

参考文献 **67**
最重要文献 67
[Aurora19] 67
[Virgo20] 67
背景文献 67
[IOP16] 67
[FRI18] 68
[GKR08] 68
後続研究・注意喚起 68
[VirgoRevisit25] 68
[FSFRI23] 69
周辺文献（比較や発展のために） 69
[Libra19] 69
[Spartan20] 69
[Fractal20] 69
読み進め方の提案 70
最後に 70

まえがき

本書は、透明なゼロ知識証明系の代表例として **Aurora** と **Virgo** を学ぶための入門書です。対象読者は、ゼロ知識証明に興味がある初学者・学部生・修士学生、そして「FRI 系の透明証明」と「GKR 系の透明証明」を体系的に整理したい読者です。

ただし、本書は原論文の完全再現ではありません。Aurora と Virgo の原論文は、いずれも高度に圧縮された専門文献であり、低次数多項式、符号理論、和検査、オラクルアクセス、Fiat–Shamir 変換といった要素を当然知っているものとして書かれています。そこで本書では、**厳密さを落とし過ぎず、しかし読み手の認知負荷を下げる**ことを最優先にしました。

この本の狙い

Aurora と Virgo を並べて学ぶと、透明 zk 証明の設計思想がかなりはっきり見えてきます。

- Aurora は、**R1CS を Reed–Solomon 符号語と低次数テストの世界へ持ち込む**タイプの設計です。
- Virgo は、**layered arithmetic circuit を GKR で圧縮し、それを透明な zkVPD で短い証明へ落とし込む**タイプの設計です。

どちらも「trusted setup が要らない」「ハッシュ関数中心で設計できる」「ポスト量子安全性が plausibly 成り立つ」という大きな魅力を共有します。しかし、内部で使う多項式の見方、得意な計算、検証者のボトルネックはかなり違います。そこを曖昧なまま理解すると、「どちらも透明 SNARK ですよね」で思考停止してしまいます。本書は、その停止を避けるための本です。

本書が重視すること

本書では、次の 3 点を強く意識して説明します。

1. **概念の位置づけ**
   - R1CS とは何か。
   - layered circuit とは何か。
   - なぜ低次数多項式が現れるのか。
   - IOP、FRI、GKR、VPD がどこで効くのか。
2. **直感と形式の往復**
   - まず「何をしたいのか」を平易に言う。
   - 次に、その願いがどの数式・多項式恒等式・オラクル検査に対応するかを示す。
3. **適用可能性の見取り図**
   - Aurora はどんな計算に向くのか。
   - Virgo はどんな回路で強いのか。
   - 実装や研究で何に気をつけるべきか。

読者に求める前提

望ましい前提は次の通りです。

- 有限体 $\mathbb{F}$ の記号に抵抗がない。
- 線形代数の基本（ベクトル、内積）が分かる。
- グラフとしての回路、計算量 $O(\cdot)$ の表記を読める。
- 暗号学の専門知識はなくてもよい。

逆に、次を知らなくても大丈夫です。

- FRI の厳密な健全性解析
- GKR の完全な導出
- IOP の形式定義の細部
- Fiat–Shamir の量子ランダムオラクルモデルでの安全性証明

本書の役目は、そうした高度な文献に進む前に、**地図**を渡すことです。

本書の読み方

読み方は 2 通りあります。

まず全体像を掴みたい人

次の順で読むとよいです。

1. 第 1 章「ゼロ知識証明の基礎」
2. 第 2 章「Aurora / Virgo を理解するための下準備」
3. 第 3 章「Aurora の全体像」
4. 第 5 章「Virgo の全体像」
5. 第 7 章「Aurora と Virgo の比較」
6. 第 8 章「ユースケース」

数式も含めて腰を据えて学びたい人

最初から順に読み、Aurora の手続きと Virgo の手続きを対比しながら読むのがおすすめです。特に、

- Aurora では「**univariate な見方**」
- Virgo では「**multilinear / hypercube な見方**」

という差が核心です。ここを意識すると、後続の多くの透明証明系も整理しやすくなります。

記号と約束

本書では、次の記号をよく使います。

記号	意味
x	公開される statement
w	秘密の witness

記号	意味
$R(x, w) = 1$	statement と witness が関係 R を満たすこと
$\mathbb{F}$	計算を行う有限体
C	回路の gate 数
D	回路の depth
n	文脈依存のサイズ。原論文の式を引用する際は元の記号を尊重する
$\tilde{f}$	関数 f の多項式拡張（とくに multilinear extension など）

重要なのは、**原論文ごとにサイズ記号の流儀が微妙に異なる**ことです。本書では、複雑性を引用する場面ではできるだけ原論文の書き方に合わせ、混乱しそうなら文章でも補います。

先に一言でいうと

Aurora も Virgo も、「大きな計算を、小さな数のランダム点評価の検査へ圧縮する」プロトコルです。違いは、その圧縮をどの幾何学で見るかです。

- Aurora は「大きな表を **低次数な univariate 多項式の評価表** として見る」。
- Virgo は「大きな表を **Boolean hypercube 上の関数の multilinear extension** として見る」。

この視点の違いが、プロトコル設計・検証時間・実装の難しさ・向く回路の形にまで効いてきます。

文献上の事実と本書の解釈

本書では、できるだけ原論文に基づいて記述しますが、教科書として読みやすくするために、

- 直感的なたとえ
- 簡略化した模式図
- 設計上の助言
- 「どちらが向くか」という判断基準

も併せて書きます。

このうち、**複雑性・定義・主張の本体**は参考文献に挙げた原著論文を根拠にし、**ユースケースの整理や実装上の推奨**は本書の解釈として明示的に述べます。とくに第 8 章と第 9 章では、その区別を意識して読み進めてください。

本書で使う主要文献

本書の中心文献は次の 2 本です。

- [Aurora19] Eli Ben-Sasson, Alessandro Chiesa, Michael Riabzev, Nicholas Spooner, Madars Virza, Nicholas P. Ward, *Aurora: Transparent Succinct Arguments for R1CS*, EUROCRYPT 2019.

- [Virgo20] Jiaheng Zhang, Tiancheng Xie, Yupeng Zhang, Dawn Song, *Transparent Polynomial Delegation and Its Applications to Zero Knowledge Proof*, IEEE Symposium on Security and Privacy 2020.

これに加えて、背景として次も重要です。

- [IOP16] Interactive Oracle Proofs
- [FRI18] Fast Reed–Solomon Interactive Oracle Proofs of Proximity
- [GKR08] Delegating Computation: Interactive Proofs for Muggles

ここから先の心構え

Aurora と Virgo は、いきなり実装から触ると「用語だけ多くて、何が嬉しいのか分からない」と感じやすい題材です。逆に、概念だけで済ませると「低次数性・符号語・sumcheck・GKR が、結局どこで必要なのか」が分からなくなります。

したがって、本書ではいつも次の問いを持ちながら進みます。

1. いま検査したい対象は何か。
2. それをなぜ多項式に直すのか。
3. なぜ少数のランダム検査で済むのか。
4. なぜ witness が漏れないのか。
5. なぜ trusted setup が不要なのか。

この 5 つを見失わなければ、Aurora と Virgo はかなり見通しよく理解できます。

第 1 章

ゼロ知識証明の基礎

この章では、Aurora と Virgo に入る前に、ゼロ知識証明そのものの基礎を確認します。ここで大事なのは、「**何を、誰に、どこまで伝えるのか**」です。Aurora も Virgo も、結局はこの一般枠組みの上に乗っています。

1.1 ゼロ知識証明とは何か

ゼロ知識証明の典型的な状況は次です。

- 証明者（prover）は、ある秘密 w を知っている。
- 検証者（verifier）は、その秘密の中身は知りたくない。
- しかし、秘密 w が公開情報 x に対して正しいこと、すなわち $R(x, w) = 1$ であることだけは納得したい。

ここで R は「関係（relation）」です。たとえば、

- x: 「公開鍵 pk」
- w: 「秘密鍵 sk」
- $R(x, w) = 1$: 「sk から pk が導かれる」

という形でもよいですし、

- x: 「ある計算の入力と出力」
- w: 「計算途中の内部状態や秘密入力」
- $R(x, w) = 1$: 「その計算が本当に正しい」

という形でもよいです。

Aurora と Virgo は後者、つまり **計算が正しく行われたことの証明** に属します。

1.2 3 つの基本性質

ゼロ知識証明では、通常次の 3 性質を考えます。

1.2.1 完全性（completeness）

正直な証明者が正しい witness を持っているなら、正直な検証者は受理する。

これは「本当に正しい人がちゃんと通る」ことです。

1.2.2 健全性（soundness）

もし statement が偽なら、どんな不正な証明者でも高確率で検証者を騙せない。

これは「嘘つきが通らない」ことです。

1.2.3 ゼロ知識性（zero knowledge）

検証者は、受理するかどうか以上の本質的情報を witness について得ない。

これは「通った事実は分かるが、中身は分からない」ことです。

この 3 つのうち、Aurora と Virgo でとくに難しいのは後ろ 2 つです。

- 健全性のためには、少数のランダム検査だけで大きな計算全体を縛る必要がある。
- ゼロ知識性のためには、検査に必要な情報だけを見せ、witness の構造を隠す必要がある。

1.3 直感的な例：年齢証明

酒類購入の場面を考えます。

- 検証したい事実: 「あなたは 20 歳以上である」
- 見せたくない情報: 正確な生年月日、氏名、住所

このとき理想は、「20 歳以上である」ことだけを示し、それ以外は隠すことです。ゼロ知識証明は、これを計算一般に拡張した道具だと思えばよいです。

ただし Aurora / Virgo の世界では、証明対象は年齢ではなく **算術回路の正しさ** です。年齢証明のような命題も、最終的には加算・乗算・比較の回路へ落とせれば扱えます。

1.4 証明したいのは「知識」か「計算」か

ゼロ知識証明というと「秘密を知っていることの証明」という印象が強いですが、実務や研究でとても重要なのは **計算の正しさの証明** です。

たとえば次のような場面です。

- 秘密入力を使って機械学習推論をした。その出力が正しいことを証明したい。
- 秘密の取引履歴を見せずに、残高が負でないことを証明したい。
- サーバに計算を委託し、返ってきた結果が本当に正しいことを確認したい。
- ブロックチェーン上で、多数の取引をまとめて処理したことを短い証明で示したい。

ここで必要なのは「秘密そのもの」ではなく、「**秘密を使った計算の正しさ**」です。Aurora と Virgo は、この用途に向いた一般目的の zk 証明系です。

1.5 関係 R と NP 的な見方

暗号文献ではしばしば、証明したい対象を

$$R(x, w) = 1$$

という形で表します。ここで、

- x: 公開 statement
- w: 秘密 witness
- R: 多項式時間で検査できる関係

です。ゼロ知識証明は、「w を明かさずに、ある w が存在することを示す」ものだと理解できます。

計算を証明したい場合は、計算内容を回路や制約系に変換して、その制約を満たす内部値列を witness とみなします。つまり、

- statement = 「回路の仕様」「公開入力」「公開出力」
- witness = 「秘密入力」「中間 wire 値」
- relation = 「回路の各ゲートが正しく評価されている」

という構図です。

1.6 計算はなぜ回路にするのか

Aurora や Virgo では、最終的に証明したい計算を **算術回路** に変換します。算術回路とは、有限体 $\mathbb{F}$ 上で

- 足し算
- 掛け算
- 定数倍

だけで構成された回路です。

なぜ回路へ変換するのでしょうか。理由は単純で、暗号プロトコルが得意なのは「自由なプログラム」そのものではなく、**局所的な代数制約** だからです。
回路にすると、各ゲートは非常に単純な等式に分解されます。

- 加算ゲートなら $c = a + b$
- 乗算ゲートなら $c = a \cdot b$

この「局所性」が、後で多項式恒等式にまとめられ、ランダム検査で圧縮されます。

1.7 小さな具体例

次の式を考えます。

$$z = xy + x + 2$$

公開 statement は「$z = 20$」。
秘密 witness は「$x = 3, y = 5$」。

このとき

$$3 \cdot 5 + 3 + 2 = 20$$

なので、確かに正しい witness があります。

ここで重要なのは、検証者は $x = 3, y = 5$ を知る必要がないことです。検証者が知りたいのは、**ある x, y が存在して、その式を満たす**という事実だけです。

この例は小さすぎて、普通に式を展開すれば終わります。しかし、Aurora や Virgo が扱うのは数百万ゲート級の回路です。そのとき「全部を再計算する」のではなく、「少数のランダム検査だけで、全体が正しいと信じられる」仕組みが必要になります。

1.8 なぜ「短い証明」が欲しいのか

単に正しさを示すだけなら、witness 全体を提出してしまえばよいように思えるかもしれません。しかしそれでは問題があります。

1. **プライバシーがない**
   witness を見せたらゼロ知識ではありません。
2. **通信量が大きい**
   大規模回路の中間値は膨大です。
3. **検証が重い**
   検証者が全部再計算したら、証明する意味が薄れます。

そこで、現代的な zk 証明系では次を目指します。

- 証明は短い
- 検証は速い
- witness は隠れる
- できれば trusted setup も不要

Aurora と Virgo は、まさにこの設計目標に向けて作られました。

1.9 SNARK, SNARG, STARK という言葉

文献には似た言葉がたくさん出ます。初心者が混乱しやすいので、ここでざっくり整理します。

1.9.1 SNARK / SNARG

- **S**uccinct: 証明や検証が短い / 速い
- **N**on-interactive: 非対話的（1 本の証明で済む）
- **A**rgument: 計算量制限された証明者に対する健全性
- **R/K**: of Knowledge の K が付く流儀もある

厳密には論文ごとに語の使い分けがあります。本書では、細部が本質でないところでは「短い zk 証明系」として読み進めます。

1.9.2 STARK

一般には、

- transparent setup
- ハッシュ中心
- FRI ベース
- 大きめだがポスト量子安全性が期待できる

というイメージで理解されることが多いです。ただし、Aurora は FRI に近い透明系でありながら、歴史的には STARK と完全には同一視されません。重要なのは名称よりも**設計の骨格**です。

1.10 trusted setup と transparent setup

ゼロ知識証明系の多くは、最初に共通参照文字列（CRS）や構造化参照文字列（SRS）を生成する setup を必要とします。これが trusted setup です。

問題は、その setup に不正があると、後で偽の証明を作れる可能性があることです。したがって、trusted setup を使う系では、マルチパーティセレモニーやアップデート可能な SRS など、追加の工夫が要ります。

これに対し Aurora と Virgo は **transparent** です。
つまり、秘密の trapdoor を含む初期設定を前提にしません。使うのは主に

- ハッシュ関数
- 有限体上の演算
- 符号理論的な低次数検査

です。

この性質は、公的インフラやオープンな検証環境において大きな魅力になります。

1.11 でも「transparent なら常に勝ち」ではない

ここは重要です。transparent setup は強い長所ですが、他の指標もあります。

- 証明サイズ
- 証明生成時間
- 検証時間
- 実装の複雑さ
- 使いたい回路表現との相性

たとえば、trusted setup を許せば非常に短い証明や高速検証が得られる場合もあります。したがって、「transparent だから自動的に最善」とは限りません。Aurora と Virgo のどちらを採るかは、**何を証明したいか**と**どんな実装制約があるか**で決まります。

1.12 この本での設計観

本書では、ゼロ知識証明を次の 4 段階で見ることにします。

1. **計算を制約へ変換する**
2. **制約を多項式的な主張へ変換する**
3. **巨大な主張をランダム検査で圧縮する**
4. **その過程で witness を隠す**

Aurora も Virgo も、この 4 段階の具体化です。違うのは 2 と 3 の実装方式です。

- Aurora は R1CS を低次数なコード語へ埋め込み、univariate な検査へ寄せる。
- Virgo は layered circuit を GKR でたどり、multilinear な評価主張へ寄せる。

この差を意識して読むと、単なる暗記ではなく設計原理として理解できます。

1.13 この章のまとめ

この章の要点は次の通りです。

- ゼロ知識証明は「正しさだけを示し、中身を漏らさない」ための道具である。
- Aurora / Virgo が扱うのは、主に「計算の正しさ」の証明である。
- 計算は算術回路や制約系へ変換される。
- 良い zk 証明系は、短い証明・速い検証・秘匿性・透明性のバランスを取る。
- Aurora と Virgo は、その透明な実現方法として非常に重要である。

次章では、Aurora と Virgo を読むために必須の下準備 ——R1CS、layered circuit、低次数多項式、IOP、FRI、GKR—— を丁寧に整えます。

第 2 章

Aurora / Virgo を理解するための下準備

この章では、Aurora と Virgo に共通する基礎をまとめます。ここを押さえると、後の章で出てくる

- R1CS
- layered circuit
- Reed–Solomon 符号
- IOP
- sumcheck
- FRI
- GKR

が、ばらばらの専門用語ではなく、1 本の設計線上に並んで見えるようになります。

2.1 算術回路とは何か

算術回路は、有限体 $\mathbb{F}$ 上での計算を表す DAG（有向非巡回グラフ）です。各ノードは

- 加算
- 乗算
- 定数
- 入力

のいずれかです。

たとえば、

$$z = xy + x + 2$$

という式は、次のように分解できます。

1. $a = x \cdot y$
2. $b = a + x$
3. $z = b + 2$

ゲート単位で見ると、各条件は局所的です。
この「局所的な制約」を大量に並べると、大きな計算全体を表せます。

2.2 layered arithmetic circuit

Virgo が特に好むのは **layered arithmetic circuit** です。
これは、回路を層ごとに並べて、各層のゲートが次の層のゲートからのみ入力を受ける形に整えたものです。

イメージは次の通りです。

- 最下層: 入力

- 中間層: 加算ゲート・乗算ゲート
- 最上層: 出力

layered にすると、計算の正しさを「上の層から下の層へ」順にたどるプロトコルを設計しやすくなります。GKR はまさにこの形を利用します。

2.2.1 なぜ layer 化がうれしいのか

layer 化の利点は、「ある層の値が正しいかどうか」を、その一つ下の層の値に帰着できる点です。
つまり、

- layer i の主張
- $\Downarrow$
- layer $i+1$ の主張
- $\Downarrow$
- $\cdots$
- $\Downarrow$
- 最後は入力の主張

という形で、大きな計算を段階的に縮約できます。

2.3 R1CS とは何か

Aurora の中心表現は **R1CS (Rank-1 Constraint Satisfaction)** です。

R1CS では、各制約を

$$\langle A_i, z\rangle \cdot \langle B_i, z\rangle = \langle C_i, z\rangle$$

という形で書きます。ここで

- z: 定数 1、公開入力、秘密入力、中間値を並べたベクトル
- A_i, B_i, C_i: 線形形式を表す係数ベクトル

です。

2.3.1 例：$z = xy + x + 2$ を R1CS にする

変数ベクトルを

$$\mathbf{z} = (1, x, y, a, z)$$

としましょう。ここで $a = xy$ です。

すると、次の 2 制約で十分です。

制約 1

$$x \cdot y = a$$

これは

$$\langle A_1, \mathbf{z}\rangle = x, \quad \langle B_1, \mathbf{z}\rangle = y, \quad \langle C_1, \mathbf{z}\rangle = a$$

に対応します。

制約 2

$$(a + x + 2) \cdot 1 = z$$

これは

$$\langle A_2, \mathbf{z} \rangle = a + x + 2, \quad \langle B_2, \mathbf{z} \rangle = 1, \quad \langle C_2, \mathbf{z} \rangle = z$$

に対応します。

このように、回路の局所条件を「線形形式どうしの積 = 線形形式」という形にまとめるのが R1CS です。

2.3.2 なぜ R1CS が便利なのか

R1CS の良さは、制約が一様な形になることです。
どんな複雑な回路でも、最終的には

- 線形結合を作る
- それどうしを掛ける
- 右辺の線形結合と一致させる

という同じ型の制約に落ちます。Aurora はこの一様性を、多項式恒等式の形へ変換して扱います。

2.4 なぜ多項式が出てくるのか

ここは Aurora / Virgo の核心です。

大雑把に言うと、**巨大な表を、そのまま扱うのは重い。だから多項式の評価表だと思って扱う**のです。

多項式には強い「剛性」があります。
次数 d の多項式は、$d + 1$ 個より多くの点で一致すれば、もはや同じ多項式です。
この剛性のおかげで、

- 本当に低次数なものか
- 一部だけ都合よく改ざんされていないか

を、少数のランダム検査で高確率に検出できます。

2.4.1 低次数性の直感

たとえば 1 次多項式 $f(X) = 3X + 2$ を $\mathbb{F}_{17}$ 上で考えます。
領域 $H = \{1, 2, 3, 4\}$ 上での評価は

$$(5, 8, 11, 14)$$

です。

もし誰かが 1 箇所だけ改ざんして

$$(5, 8, 12, 14)$$

を「同じくらい自然な表です」と主張しても、これは元の 1 次多項式の評価表ではありません。
「表として」見ると 1 箇所の違いに過ぎませんが、「低次数多項式の評価表」として見ると非常に不自然です。
このギャップを利用するのが低次数テストです。

2.5 Reed–Solomon 符号

Reed–Solomon 符号は、「低次数多項式をある評価領域で評価したベクトル全体」のことです。

- ある次数上限 d
- ある評価領域 $H \subseteq \mathbb{F}$
- 多項式 f の次数 $< d$

を決めると、コード語は

$$(f(\alpha))_{\alpha \in H}$$

です。

重要なのは、**符号語には冗長性がある**ことです。
この冗長性のおかげで、「たまたま少数点だけ合っている偽物」を見破りやすくなります。

Aurora はこの Reed–Solomon 的な見方を強く使います。

2.6 関数の拡張：multilinear extension

Virgo を理解するには、もう一つ別の見方が要ります。
それが **multilinear extension** です。

関数

$$V : \{0,1\}^\ell \to \mathbb{F}$$

が与えられたとき、これを Boolean hypercube の外へ自然に延長した多項式 $\tilde{V}$ を multilinear extension と呼びます。

$$\tilde{V}(x_1, \dots, x_\ell) = \sum_{b \in \{0,1\}^\ell} V(b) \prod_{j=1}^{\ell} ((1 - x_j)(1 - b_j) + x_j b_j).$$

意味は単純です。

- 元の表 V は $\{0,1\}^\ell$ 上だけで定義されている。
- それを「各変数について次数 1」の多項式へ拡張する。
- Boolean 点では元の値を保つ。

Virgo / GKR は「各 layer の gate 値表」を、この multilinear extension として見ます。

2.7 IOP とは何か

Aurora を読むと **IOP (Interactive Oracle Proof)** が出てきます。
これは interactive proof と PCP 的な発想の中間にあるモデルです。

平たく言えば、

- 証明者は長いメッセージを送る
- ただし検証者はその全部を読む必要はない
- 検証者は、その長いメッセージを「配列へのオラクルアクセス」として扱い、必要箇所だけランダムに読む

というモデルです。

2.7.1 なぜ IOP がうれしいのか

巨大な証明をまるごと読むのは重いですが、もしその証明が

- よく構造化されていて
- 少数点の検査だけで大域的正しさが言えれば

検証者はごく一部しか見なくて済みます。Aurora はこの IOP 的世界観の上で設計されています。

2.7.2 非対話化はどうするのか

IOP は本来対話的です。しかし実用上は 1 本の証明にしたい。
そのためによく使うのが

- **Merkle 木**で長いオラクルをコミットする
- **Fiat–Shamir 変換**で検証者のランダム性をハッシュから導く

という定石です。

すると、証明者は

1. オラクル列にコミットし、
2. ハッシュでチャレンジを決め、
3. 必要箇所の開示と認証パスを付ける

ことで、非対話証明にできます。

2.8 sumcheck の直感

Aurora と Virgo の両方で、**sumcheck 的発想**が極めて重要です。

最も簡単な 2 変数版を見ます。
多項式 $g(X_1, X_2)$ について

$$T = \sum_{b_1, b_2 \in \{0,1\}} g(b_1, b_2)$$

という主張を検証したいとします。

2.8.1 1 ラウンド目

証明者は

$$s_1(X) = g(X, 0) + g(X, 1)$$

を送ります。検証者は

$$s_1(0) + s_1(1) = T$$

を確認します。

2.8.2 2 ラウンド目

検証者がランダム r_1 を選び、証明者は

$$s_2(X) = g(r_1, X)$$

を送ります。検証者は

$$s_2(0) + s_2(1) = s_1(r_1)$$

を確認します。

2.8.3 最後

検証者がランダム r_2 を選び、最後に

$$s_2(r_2) = g(r_1, r_2)$$

を確認すればよい。

こうして、「4 点の総和」の検査が「ランダム点での 1 回の評価」に圧縮されました。
高次元ならこの考えを何ラウンドも繰り返します。

2.8.4 なぜこれが効くのか

不正な証明者が途中で偽の多項式を送ると、どこかで次数や整合性が崩れます。ランダム点 r_i での検査は、その崩れを高確率で露呈させます。

- Aurora では **univariate 版の和検査**
- Virgo/GKR では **multilinear な和検査**

が登場します。

2.9 FRI の直感

FRI (Fast Reed–Solomon Interactive Oracle Proofs of Proximity) は、ざっくり言えば

「与えられた巨大な表が、本当に低次数多項式の評価表に近いか」

を効率よく確かめるプロトコルです。

2.9.1 何をしたいのか

証明者が巨大なオラクル表 f を出してきたとき、検証者は全部を読みたくありません。
でも「これは低次数多項式の評価表です」と言われた以上、その主張が本当かを確かめたい。

そこで FRI は、表を段階的に「折りたたむ」ことで、問題をどんどん小さくします。
低次数なら、折りたたんだ後も低次数性が保たれます。偽物なら、どこかで整合性が崩れます。

2.9.2 直感的な折りたたみ

FRI の詳細な式はやや技巧的ですが、感覚としては次のようなものです。

1. 大きな評価表を取る。
2. ランダム係数を使って、表のペアを混ぜ合わせる。
3. それに対応する「半分くらい小さい」表を作る。
4. これを繰り返して、最後は非常に小さな問題にする。

各段階で検証者は少数点だけを見るので済みます。Aurora では、この FRI が低次数性チェックの中核になります。

2.10 GKR の直感

GKR（Goldwasser–Kalai–Rothblum）プロトコルは、layered circuit の計算正しさを効率的に検証するための対話証明です。

基本のアイデアは、

> 「各 layer の全ゲート値を 1 個ずつ確かめるのではなく、その layer 全体の整合性を、1 本の和検査で圧縮する」

というものです。

2.10.1 layer ごとの値表

layer i の各ゲート値を並べた表を V_i とします。
GKR は、この表を multilinear extension $\tilde{V}_i$ として扱います。

検証者は「layer i の全ゲートが正しいか」を直接見る代わりに、ランダム点 r_i における $\tilde{V}_i(r_i)$ の主張へ圧縮します。

すると、次はその主張を「ひとつ下の layer の主張」へ帰着できます。これを繰り返せば、最終的に入力 layer まで降りていきます。

2.10.2 wiring predicate

なぜ 1 つ下に降りられるのでしょうか。
それは、各ゲートが

- 加算ゲートか
- 乗算ゲートか
- どの 2 つの子ゲートに接続しているか

という wiring 情報を持つからです。

この wiring を表す述語が efficient に計算できると、各 layer の正しさを「次 layer の値表」との関係として低次数多項式で表現できます。Virgo では、この GKR 的縮約が backbone になります。

2.11 「structured circuit」とは何か

Virgo の複雑性で出てくる「structured circuit」は、初心者がつまずきやすい語です。
直感的には、

> ゲートのつながり方が規則的で、検証者が wiring を速く計算できる回路

と理解すれば十分です。

典型例は、

- 行列演算
- FFT
- Merkle 木
- 各層がほぼ同じ形を繰り返すデータ並列回路

などです。

逆に、接続がぐちゃぐちゃで、検証者が「その gate はどことつながるのか」を毎回重く計算しないといけない回路では、Virgo の verifier succinctness の旨味が薄れます。

2.12 Aurora と Virgo の違いを先取りする

ここまでの準備を使うと、両者の違いをかなり短く言えます。

2.12.1 Aurora 的世界観

- 計算を R1CS にする
- それを低次数なコード語の検査へ変換する
- IOP + FRI で低次数性を保証する
- 少数点の開示で全体の正しさを主張する

2.12.2 Virgo 的世界観

- 計算を layered circuit にする
- layer ごとの値表を multilinear extension として扱う
- GKR で上の layer の正しさを下の layer へ縮約する
- 多項式評価の主張を zkVPD で短く証明する

前者は **univariate / Reed–Solomon / FRI**、後者は **multilinear / GKR / zkVPD** です。

2.13 この章のまとめ

この章の要点を整理します。

- 算術回路は、大きな計算を加算・乗算の局所制約に分解するための表現である。
- R1CS は Aurora 側の標準表現であり、制約を $\langle A_i, z\rangle \cdot \langle B_i, z\rangle = \langle C_i, z\rangle$ に揃える。
- layered circuit は Virgo 側の自然な表現であり、layer ごとに正しさを縮約しやすい。
- 低次数多項式は「剛性」が高く、少数のランダム検査と相性がよい。
- Reed–Solomon 符号は Aurora の視点、multilinear extension は Virgo / GKR の視点である。
- IOP は「長い証明を全部読まない」ためのモデルであり、Merkle + Fiat–Shamir で非対話化できる。

- sumcheck は「大量の和の主張をランダム点評価へ圧縮する」道具である。
- FRI は低次数性を効率よく検査する。
- GKR は layer ごとに回路の正しさを縮約する。

次章から、まず Aurora を詳しく見ます。

第 3 章

Aurora の全体像

この章では、Aurora を「何を狙って作られた系なのか」「どの部品がどの役割を担うのか」という観点から俯瞰します。細部よりも、まず設計思想を掴む章です。

3.1 Aurora とは何か

Aurora は、Ben-Sasson, Chiesa, Riabzev, Spooner, Virza, Ward による透明な zk 証明系です。[Aurora19]
原論文の要約を一言で言えば、

> **R1CS に対する、transparent setup・軽量暗号・FRI ベースの succinct zk 証明**

です。

原論文が強調する性質は次の通りです。

- 対象は **R1CS**
- **transparent setup**
- **plausibly post-quantum secure**
- **lightweight cryptography**
- 証明サイズは $O(\log^2 n)$（n は制約数）
- 証明生成は $O(n \log n)$ field operations
- 検証は $O(n)$

ここでまず気づくべきことは、「proof size は非常に短いが、verifier time は完全な定数・対数ではなく線形である」という点です。これは Aurora の性格を理解する上で重要です。

3.2 Aurora が解きたい問題

Aurora が念頭に置くのは、一般の計算を R1CS へ変換したあと、その satisfiable 性を **短い証明**で示したいという問題です。

より具体的には、次を同時に満たしたいわけです。

1. trusted setup が不要
2. pairing のような重い公開鍵暗号的構造に依存しない
3. 証明はできるだけ短い
4. witness は漏れない
5. できればポスト量子でも期待が持てる

この設計目標は、当時の zkSNARK 系列の中でかなり鮮明でした。Aurora は、STARK に近い透明性と、SNARK 的な短い証明を同時に狙う設計点にあります。

3.3 Aurora の基本アイデア

Aurora の大筋は次の 4 段階です。

1. **R1CS を、多項式やコード語の整合性チェックへ変換する**
2. **その整合性を、少数の algebraic checks に圧縮する**
3. **オラクルが本当に低次数なコード語に対応していることを FRI で確認する**
4. **オラクル全体は Merkle 木へコミットし、必要な点だけ開示する**

つまり Aurora は、

- 「制約の正しさ」と
- 「開示された表が本当に低次数多項式の評価表であること」

を同時に押さえる設計です。

3.4 なぜ R1CS から多項式へ行くのか

R1CS はもともと「各制約ごとの局所的な等式」です。
しかし、そのままだと制約数が多いと検査が重い。そこで Aurora は、制約列を低次数多項式の言葉へ埋め込みます。

直感的には、

- witness ベクトルや補助情報を大きな表として並べる
- その表を「低次数多項式の評価表」とみなす
- 制約違反があるなら、それはある多項式恒等式の破れとして現れる

という流れです。

ここで「低次数」であることが非常に大事です。低次数でなければ、証明者は都合のよい値だけを局所的に貼り合わせた“偽物の表”を作れてしまいます。そこで Aurora では、FRI を用いて **低次数性の保証** を本気で入れます。

3.5 Aurora を構成する主要部品

Aurora を、部品ごとに分けて見ましょう。

3.5.1 1. R1CS

入力となる計算はまず R1CS に落とされます。
Aurora はこの R1CS が satisfiable であること、すなわち witness が存在することを証明します。

3.5.2 2. Encoded IOP

Aurora は IOP 的な枠組みで動きます。[IOP16]
ただし単なる IOP ではなく、Reed–Solomon 的な符号化を前提とする「encoded」な見方が重要です。証明者の長いメッセージは、低次数多項式の評価表として扱われます。

3.5.3 3. Univariate sumcheck の新しい IOP

原論文の key ingredient として特に強調されるのが、**classical sumcheck の univariate analogue** です。[Aurora19]

通常の sumcheck は、多変数多項式を Boolean hypercube 上で足し上げる形を扱います。Aurora ではこれと似た精神を、Reed–Solomon 符号語の和に対して実現します。
この設計が、Aurora を「R1CS + RS codeword + FRI」の線へきれいに乗せています。

3.5.4 4. FRI

FRI は、開示されたオラクルが本当に低次数多項式の評価表に近いかを調べる低次数テストです。[FRI18]
Aurora では、この FRI が soundness のかなり深い部分を支えます。

ここでの直感はシンプルです。

- 低次数多項式の表は、ランダムな折りたたみや点検査に対して整合的に振る舞う。
- 偽物の表は、どこかでその整合性を失う。

3.5.5 5. Merkle コミットメントと Fiat–Shamir

IOP を非対話化するには、オラクル全体をハッシュで固定する必要があります。
Aurora は典型的に、オラクルを Merkle 木でコミットし、チャレンジを Fiat–Shamir で導きます。

これにより、証明は

- いくつかの根ハッシュ
- 必要位置の値
- 認証パス
- FRI 関連の開示情報

の集合として表現できます。

3.6 Aurora の複雑性をどう読むか

原論文の代表的な複雑性は次です。

- proof size: $O(\log^2 n)$
- prover: $O(n \log n)$
- verifier: $O(n)$

ここで n は制約数です。

3.6.1 proof size が短いとはどういうことか

制約数が数百万でも、証明サイズは polylog に抑えられます。
これは透明系としてはかなり魅力的です。

原論文では、128-bit security で「数百万制約でも 130kB 未満」という具体値が示されています。[Aurora19]
この数値は、Aurora が「透明なのに証明が短い」という印象を広く与えた理由の一つです。

3.6.2 verifier が $O(n)$ なのは弱点なのか

ここは初心者が誤解しやすい点です。

一見すると、検証が線形なら“succinct ではない”ように見えるかもしれません。しかし、少なくとも原論文の主張の文脈では、検証者は statement の記述自体を読む必要があります。R1CS が公開 statement の一部なら、それを読むコストはある程度避けられません。

ただし実務の感覚では、

- Groth16 のような非常に短い verifier
- 一部の preprocessing 系

と比べると、Aurora verifier は軽いとは言っても“極小”ではありません。
したがって Aurora は、**proof size の短さと transparent setup のバランスが強い**設計と見るのが自然です。

3.7 Aurora の何が新しかったのか

Aurora の歴史的な価値は、単に一つのプロトコルができたことだけではありません。少し抽象化すると、Aurora は次を示しました。

1. R1CS ベースでも、transparent な zk 証明をかなり短くできる
2. IOP と RS code と FRI の組合せは実用的設計線になりうる
3. hash ベースの部品だけでも、かなり強い性能を狙える

つまり Aurora は、「透明な zk 証明は proof size が大きい」という当時の固定観念をかなり押し広げた系でした。

3.8 Aurora の直感的な図式

Aurora の流れを、細部を捨てて模式化するとこうなります。

```
計算
  ↓
R1CS
  ↓
R1CS の制約を多項式的主張へ変換
  ↓
その主張を支えるオラクル群を構成
  ↓
オラクルが低次数コード語であることを FRI で確認
  ↓
Merkle + Fiat-Shamir で非対話化
  ↓
短い zk 証明
```

重要なのは、Aurora が「R1CS を直接 bit-by-bit に見る」のではなく、**一度、符号化された多項式の世界へ持ち上げる**ことです。

3.9 Aurora が向く場面

本書の整理として、Aurora は次のような場面で発想上とても自然です。

- すでに R1CS へ落とすパイプラインがある
- transparent setup が重要
- pairing に依存したくない
- proof size をできるだけ抑えたい
- 低次数テストや FRI 系の思想を中心に据えたい

逆に、検証者ができるだけ軽くあってほしい場面や、回路が非常に規則的で GKR 的縮約が効く場面では、Virgo 的な発想がより魅力的になることがあります。

3.10 Aurora と STARK の関係をどう見るか

初心者は Aurora を「STARK の一種ですか」と聞きたくなるかもしれません。
厳密な分類論争に入る必要はありませんが、理解の助けとしては次のように考えるとよいです。

- Aurora は **FRI / IOP / transparent** という意味で STARK 的発想に近い
- ただし対象表現は R1CS であり、設計上の強調点も少し異なる
- 歴史的には、透明証明の設計空間を広げた重要な系として見るのがよい

本書では名称にこだわるよりも、**何をどの代数表現へ落としているか**に注目します。

3.11 Aurora を読むときの心の置き場

Aurora の原論文を読むと、最初は用語が多くて息苦しく感じます。
しかし読むべき核心は、実は次の 3 行です。

1. R1CS の satisfiability を、低次数オラクルの整合性へ翻訳する
2. その整合性を、univariate sumcheck 的な仕組みと FRI で検査する
3. オラクル全体は見せず、必要点だけ開示する

この骨格が見えれば、あとは個々の補題や多項式定義が「どの部品を支えるためのものか」として読みやすくなります。

3.12 この章のまとめ

- Aurora は、R1CS に対する transparent / hash-based / plausibly post-quantum な zk 証明系である。
- 証明サイズは $O(\log^2 n)$、証明生成は $O(n \log n)$、検証は $O(n)$。
- 核心は、R1CS を低次数多項式と Reed–Solomon 符号語の世界へ翻訳することにある。
- IOP 的に巨大なオラクルを扱い、FRI で低次数性を保証する。
- univariate sumcheck 的な考え方が Aurora の中核部品である。
- transparent で短い proof size を実現した点が、Aurora の大きな歴史的価値である。

次章では、Aurora を小さな例に沿って「何をどう証明しているのか」を手で追います。

Aurora を手で追う

この章では、Aurora の流れを小さな例で追います。
ただし最初に強調しておきますが、**ここで示すのは原論文の完全な数式展開ではなく、教育目的の模式図**です。Aurora の実際のプロトコルは、複数のオラクル、多項式恒等式、低次数検査を精密に組み合わせています。ここでは、その骨格を掴むことを目標にします。

4.1 例題

前章と同じく、次の式を証明対象にします。

$$z = xy + x + 2$$

公開 statement は $z = 20$。
秘密 witness は $x = 3, y = 5$ です。

中間値 $a = xy$ を導入すると、

$$a = xy, \qquad z = a + x + 2$$

です。

4.2 Step 1: R1CS に落とす

まず回路を R1CS にします。
変数ベクトルを

$$\mathbf{z} = (1, x, y, a, z)$$

と置くと、制約は

1. $x \cdot y = a$
2. $(a + x + 2) \cdot 1 = z$

です。

Aurora の出発点は、このような「線形形式の積 = 線形形式」の列です。

4.2.1 ここでの本質

Aurora は **各制約を 1 個ずつ検証したいわけではない**、という点が重要です。
制約数が数百万あれば、1 個ずつ見るのは無理です。
そこで Aurora は、これらの制約を一括して多項式的主張に包みます。

4.3 Step 2: 証明者が大きな表を作る

証明者は witness と補助データから、いくつかの大きな表を作ります。
直感的には、次のような情報が入っています。

- witness の符号化
- 中間値の符号化
- 制約ごとの左辺・右辺の値
- それらの整合性を束ねる補助オラクル

Aurora の重要な発想は、これらを単なる配列としてではなく、

低次数多項式をある領域で評価した表

として扱うことです。

つまり、証明者は「私はこの大きな表を持っています」ではなく、「この表はある低次数多項式に由来しています」と主張します。

4.4 Step 3: 制約違反を多項式恒等式へまとめる

各制約が正しいとは、「制約残差」が 0 であることです。
教育用に簡単化して、各制約位置 α に対して

$$Q(\alpha) = 0$$

であってほしい残差関数 Q があると思ってください。

たとえば 2 制約しかないこの例では、

- 1 本目の制約で残差 0
- 2 本目の制約で残差 0

であってほしいわけです。

Aurora の本物のプロトコルはもっと洗練されていますが、直感としては

「多数の局所制約が全部正しい」
$\Longrightarrow$
「ある多項式的な一括主張が成り立つ」

という翻訳が起きています。

4.4.1 どうして一括主張が嬉しいのか

一括主張になれば、後はそれを

- 和の主張
- 商多項式の主張
- vanishing polynomial で割り切れる主張
- 低次数性の主張

などへ変換して、ランダム検査で押さえられるからです。

Aurora の設計のうまさは、「局所制約列」から「少数の代数的主張」への翻訳にあります。

4.5 Step 4: univariate sumcheck 的な圧縮

Aurora の原論文が key ingredient として挙げるのが、**classical sumcheck の univariate analogue** です。

直感的には、検証者は

「この大きな領域全体で、残差がきちんと打ち消し合う / 0 になる」

という主張を、一点評価の主張へ圧縮したいわけです。

教育用の絵としては、

$$\sum_{\alpha \in H} Q(\alpha) = 0$$

のような形を想像するとよいです。
実際の Aurora では、Reed–Solomon 符号語上の和を扱う、より洗練された univariate チェックが使われます。

4.5.1 なぜ和を見るのか

和を取ると、多数の局所条件を 1 個の global condition にまとめられます。
そして sumcheck 的手法により、その global condition をランダム点での低次元チェックへ縮められます。

これは「大勢の生徒の答案を、1 枚ずつ読む代わりに、ランダムな集計式で一気に監査する」ようなものです。
集計式の設計が正しければ、1 人でも不正がいれば高確率で検出できます。

4.6 Step 5: でも“都合のよい表”を出されたら困る

ここで重大な問題があります。
証明者が「この表は低次数多項式の評価表です」と勝手に言っているだけなら、好きな値を貼った偽物かもしれません。

つまり Aurora で本当に検査したいのは 2 つです。

1. **代数的主張が正しいこと**
2. **その主張の入力となる表が、本当に低次数多項式から来ていること**

2 がないと、証明者は局所的にだけ辻褄の合う値を継ぎはぎして、global check をすり抜ける可能性があります。

4.7 Step 6: そこで FRI を使う

Aurora は FRI を使って、オラクルが本当に低次数であることを検査します。

FRI の直感はこうです。

1. 証明者は大きな評価表をコミットする。
2. ランダム係数を使って表を折りたたみ、より小さい表へ進む。
3. 折りたたみ前後の値が整合しているかを、少数の点で検査する。
4. 最終的に、非常に小さな表について直接確認できる段階まで落とす。

本当に低次数なら、この折りたたみ過程は整合的です。
偽物の表なら、どこかでほころびが出ます。

4.7.1 重要な直感

FRI は「点値をそのまま信用しない」ための装置です。
Aurora において FRI は、**証明者の長大なオラクル全体に対する誠実性検査**の役割を果たしています。

4.8 Step 7: オラクルは全部見せない。Merkle で固定する

Aurora は IOP 的発想の上に立っているので、証明者は巨大なオラクルを持っています。
しかし検証者は全部は見ません。

その代わり、証明者は各オラクルを

- 葉に評価値を置いた Merkle 木

でコミットし、根ハッシュだけを先に出します。

すると検証者は後から、

- 「この位置の値を見せてください」
- 「その値が本当に先ほどのコミットに属することを示してください」

と要求できます。

これにより、

- 全体は固定される
- 必要点だけ開けばよい
- 改ざんは Merkle 認証パスで防げる

という性質が得られます。

4.9 Step 8: Fiat–Shamir で非対話化する

本来 Aurora は対話的 IOP の流れを持っていますが、実用では 1 本の証明にしたい。
そこで、検証者が振るはずだったランダムコインをハッシュから導きます。

概念的には、

$$\rho_1 = H(\mathrm{root}_1, \mathrm{root}_2, \dots)$$

のようにしてチャレンジを生成し、そのチャレンジを使って次の開示位置や係数を決めます。
これを各ラウンドで繰り返すことで、対話なしで transcript 全体を固定できます。

4.10 Step 9: ゼロ知識性はどう確保するのか

ここまで聞くと、「いや、ランダム点の値をいろいろ開示したら witness が漏れないのか」と思うはずです。
その懸念は正しいです。

Aurora では、オラクルがそのまま witness の生データにならないように、**ランダムなマスキング**が施されます。
直感的には、

- 本質的な整合性は壊さない
- しかし開示される点値から witness の構造は逆算しにくい

ように、乱数で“ぼかし”を入れます。

これはゼロ知識プロトコルでよくあるパターンです。
本当に検査したい式だけは保存し、それ以外の自由度は乱数で埋めてしまうわけです。

4.11 1 本の Aurora 証明に何が入っているか

教育的に単純化すると、Aurora の非対話証明には次のようなものが入ります。

```
- 複数オラクルの Merkle root
- それに基づく Fiat-Shamir チャレンジから決まる開示値
- 各開示値の Merkle 認証パス
- 低次数性を検査する FRI 関連の開示情報
- 最終チェックに必要な少数の field 要素
```

検証者はこれらを用いて、

1. 開示値が本当にコミット済みオラクルの一部か
2. 代数的整合性が成立するか
3. FRI の整合性が成立するか

を確認します。

4.12 例題に戻ると何が起きているのか

例題 $z = xy + x + 2$ に戻ると、本質は次です。

- 証明者は $x = 3, y = 5$ を知っている。
- その witness が R1CS を満たすことを、多項式化・符号化・低次数テストで包む。
- 検証者は x も y も知らない。
- しかし、オラクル群が「正しい witness から来た低次数構造」を持つことだけは確認できる。

したがって、検証者は

「$z = 20$ に対して、確かに何らかの witness がある」

と納得できます。

4.13 Aurora の理解でつまずきやすい点

4.13.1 1. 「FRI だけで全部証明している」と思う

違います。
FRI は **低次数性の検査** です。
「制約が正しいか」という algebraic relation 自体は、別途きちんと作り込まれています。

4.13.2 2. 「R1CS をそのままランダム点で見ている」と思う

違います。
R1CS は一度、低次数多項式とコード語の世界へ翻訳されます。
Aurora の巧みさはまさにこの翻訳にあります。

4.13.3 3. 「proof size が短いなら verifier も極小だろう」と思う

そうとは限りません。
Aurora では proof size の短さと transparent setup が強い魅力ですが、verifier time は $O(n)$ です。この点は、他の設計と比較するときに重要です。

4.13.4 4. 「ゼロ知識性はハッシュしているから自動で得られる」と思う

違います。
ハッシュや Merkle 木は主にコミットメントの役割です。
ゼロ知識性そのものは、マスキングされたオラクル設計によって得られます。

4.14 Aurora を一行で再定義する

ここまでを踏まえると、Aurora は次のように言い換えられます。

> **R1CS の satisfiability を、低次数多項式の整合性主張へ変換し、その主張を IOP + FRI + Merkle + Fiat–Shamir で短いゼロ知識証明にする系**

この文を自分の言葉で言い換えられるようになると、Aurora の理解はかなり進んでいます。

4.15 この章のまとめ

- Aurora の本当の狙いは、膨大な R1CS 制約を少数の代数検査へ圧縮することにある。
- そのために、witness や補助情報を低次数多項式の評価表として扱う。
- univariate sumcheck 的発想で、大域的主張をランダム点検査へ圧縮する。
- FRI は、その表が本当に低次数であることを保証する。
- Merkle 木と Fiat–Shamir により、巨大なオラクルを非対話的な短い証明へ変換する。
- ゼロ知識性は、オラクルへの適切なマスキングで確保する。

次章では、Virgo を同様に俯瞰し、Aurora と対照的な設計思想を見ていきます。

第 5 章

Virgo の全体像

この章では、Virgo の設計思想を俯瞰します。Aurora が **R1CS + RS 符号 + FRI** の世界なら、Virgo は **layered circuit + GKR + zkVPD** の世界です。両者とも transparent ですが、内部構造はかなり違います。

5.1 Virgo とは何か

Virgo は、Zhang, Xie, Zhang, Song による透明な零知識証明系です。[Virgo20]
原論文の中心主張は、**layered arithmetic circuit に対して、trusted setup なしで succinct な零知識証明を与える**ことです。

原論文が明示する代表的な複雑性は次の通りです。

- prover time: $O(C + n \log n)$
- proof size: $O(D \log C + \log^2 n)$
- verification time: $O(D \log C + \log^2 n)$（**structured circuit なら**）
- lightweight cryptography のみを使う
- plausibly post-quantum secure

ここで

- C: ゲート数
- D: depth
- n: 入力数

です。

5.2 Virgo が解きたい問題

Virgo の狙いを Aurora と対比すると分かりやすいです。

- Aurora: 「R1CS を低次数コード語の world へ持ち込む」
- Virgo: 「layered circuit を GKR で圧縮し、その評価主張を短く証明する」

つまり Virgo は、**layered arithmetic circuit を直接のネイティブ表現として大事にする**系です。
とくに、層構造が規則的な計算に対して、GKR 由来の強みを使いたい、という意図があります。

5.3 Virgo の backbone は GKR

Virgo の最重要出発点は GKR です。[GKR08]

GKR の精神は、

> 「回路の全ゲートを 1 個ずつ検証する代わりに、各 layer の正しさを 1 つ下の layer の主張へ縮約していく」

というものです。

layer i の値表 V_i を multilinear extension $\tilde{V}_i$ として見ると、GKR は

- $\tilde{V}_i$ のランダム点での主張
- を
- $\tilde{V}_{i+1}$ のランダム点での主張

へ落としていきます。
この layer-by-layer の縮約が、Virgo の心臓部です。

5.4 でも GKR だけでは Virgo にならない

ここが大事です。
GKR は非常に美しいプロトコルですが、そのままでは「今日欲しい短い zk 証明系」にはなりません。

課題は少なくとも 3 つあります。

1. **対話的である**
2. **多項式評価の主張を効率よく扱いたい**
3. **ゼロ知識化したい**

Virgo の新しさは、GKR の縮約に対して **transparent な zero-knowledge verifiable polynomial delegation (zkVPD)** を組み合わせたことにあります。

5.5 zkVPD とは何か

VPD は、直感的には

> 「大きな多項式 f を自分で全部計算したくない verifier が、ある点 r での値 $f(r)$ を prover に“委任”し、その値が正しいと短く確かめる仕組み」

です。

zkVPD では、さらに

- その委任過程で witness に関する情報が漏れない
- 証明サイズと verification が対数的に抑えられる

ことを目指します。

Virgo の原論文が強調するのは、**Underlying Virgo is a new transparent zero knowledge verifiable polynomial delegation scheme** という点です。[Virgo20]

5.5.1 なぜ VPD が必要なのか

GKR の内部では、各 layer の値表を多項式として見たときのランダム点評価が何度も出てきます。
もしそのたびに巨大な表を全部送っていたら、通信量も verification も重すぎます。

そこで Virgo は、

- 巨大な表を多項式としてコミットし
- 必要な点での値だけを

- 短い証明付きで開示する

仕組みとして zkVPD を使います。

この意味で、zkVPD は Virgo において「GKR の圧縮結果を外に見せる窓口」の役割を持っています。

5.6 Virgo の大まかな流れ

Virgo の模式図は次のように書けます。

```
layered arithmetic circuit
  ↓
各 layer の値表を multilinear extension として見る
  ↓
GKR により layer i の主張を layer i+1 の主張へ縮約
  ↓
途中で必要になる多項式評価の主張を zkVPD で短く証明
  ↓
Merkle / ハッシュ中心の透明な非対話 proof にする
```

Aurora と並べると、違いがはっきりします。

- Aurora は「オラクル全体の低次数性」が前景
- Virgo は「layer 間の縮約と評価主張」が前景

もちろん Virgo にも低次数的・多項式的な検査はありますが、読んだ印象の中心は GKR です。

5.7 structured circuit とは何か

Virgo を語るとき、避けて通れないのが **structured circuit** です。

原論文の verifier complexity

$$O(D \log C + \log^2 n)$$

は、**回路が structured なら**という条件付きです。

5.7.1 直感的意味

structured circuit とは、おおむね

> gate の接続規則（wiring predicate）を verifier が高速に計算できる回路

です。

GKR では、各 layer について

- どの gate が加算ゲートか
- どの gate が乗算ゲートか
- どの 2 つの子へ接続しているか

という wiring 情報を多項式的に扱います。
この wiring がランダム点で速く評価できると、verifier は回路全体を読み直さずに済みます。

5.7.2 どんな回路が structured か

典型的には次のようなものです。

- Merkle 木の各層
- 行列積や畳み込みのような規則的レイアウト
- FFT のような定型 wiring
- データ並列に層が反復する回路

逆に、接続が不規則で、個々の wire をほぼ全部読まないと wiring が分からない回路では、Virgo の verifier succinctness は活きにくくなります。

5.8 Virgo の複雑性をどう理解するか

原論文の prover time は

$$O(C + n \log n)$$

です。
直感的には、「回路サイズ C にほぼ線形で、さらに入力由来の符号化コストが $n \log n$ 乗る」と考えればよいです。

5.8.1 何が嬉しいのか

GKR 系の魅力は、**大きな計算を layer ごとに圧縮できるため、prover の重さが比較的素直に回路サイズへ比例する**ことです。
Virgo はそこへ zkVPD を合わせることで、proof size と verifier も対数的に抑えようとしています。

5.8.2 verifier の見方

structured circuit なら verifier は

- depth D
- gate 数 C
- 入力数 n

に対して、polylog 的なコストでかなり短く済みます。
これは Aurora の $O(n)$ verifier と好対照です。

ただし、その代わりに回路の layer 構造や wiring の規則性が重要になります。

5.9 Virgo の concrete result

原論文の実験では、Merkle tree with 256 leaves を計算する回路について、

- proof generation: 53 秒
- verification: 50 ms
- proof size: 253 KB

という具体値が報告されています。[Virgo20]

この数値だけで全体性能を判断してはいけませんが、Virgo が

- 透明で

- 証明が数百 KB 級で
- 検証がかなり速い

という設計点にあることはよく伝わります。

5.10 Virgo の新しさを一言でいうと

Virgo の価値は、

> **GKR の layer-by-layer 圧縮を、transparent かつ zero-knowledge な多項式委任で包み、短い proof system に仕立てたこと**

にあります。

これは Aurora の「R1CS を RS codeword に埋め込む」発想とはかなり違います。Virgo はむしろ、「回路の階層構造そのものを活かし切る」設計です。

5.11 Virgo が向く場面

本書の整理として、Virgo が自然に見えるのは次のような状況です。

- 回路が layer 化しやすい
- wiring が規則的で structured
- verifier をできるだけ軽くしたい
- GKR 的な圧縮が効く
- ハッシュ中心・transparent な設計が欲しい

たとえば、

- Merkle 木
- 規則的な線形代数計算
- 同型な層が何度も繰り返される回路

などは、Virgo 的発想と相性がよいです。

5.12 Virgo のトレードオフ

Virgo にももちろん弱点や注意点があります。

5.12.1 1. layered / structured への依存

Virgo の強みは、回路が layer 化され、しかも wiring が規則的であるときに最大化します。一般の“ぐちゃぐちゃな”回路では、その良さが目減りします。

5.12.2 2. 実装と理論の距離

GKR、multilinear extension、zkVPD は、それぞれ単独でも十分難しい概念です。
したがって Virgo は、Aurora とは別の意味で「内部を理解するのに腰が要る」系です。

5.12.3 3. 後続研究による再検討

Virgo は後続研究で再検討され、zkVPD の設計・実装に関する脆弱性や制約が指摘されています。[VirgoRevisit25]

本書では第 9 章で改めて触れますが、歴史的価値とそのままの実装採用可能性は区別して読むべきです。

5.13 Aurora と見比べたときの第一印象

まだ細部に入る前ですが、ここで Aurora と Virgo の最初の対比をしておきます。

観点	Aurora	Virgo
入力表現	R1CS	layered arithmetic circuit
多項式の見方	RS codeword / univariate	multilinear extension / hypercube
中核技法	Encoded IOP + FRI	GKR + zkVPD
verifier の特徴	線形時間	structured なら polylog 的
得意な直感	低次数性の一括検査	layer-by-layer の縮約

この表だけでも、かなり性格が違うことが分かると思います。

5.14 Virgo を読むときの心の置き場

Virgo を初めて読む人は、しばしば次の 2 つを混同します。

1. GKR が何をしているのか
2. zkVPD が何をしているのか

この混同を避けるために、役割分担をはっきりさせましょう。

- **GKR**: 「回路の正しさ」を層ごとに縮約する
- **zkVPD**: 「途中で必要になる多項式評価の主張」を短く安全に証明する

この切り分けで読むと、Virgo の構造がかなり整理されます。

5.15 この章のまとめ

- Virgo は、layered arithmetic circuit に対する transparent な零知識証明系である。
- backbone は GKR であり、layer ごとの正しさを下の layer へ縮約する。
- その縮約の途中で必要になる多項式評価主張を、transparent な zkVPD で短く証明する。
- 複雑性は prover $O(C + n \log n)$、proof size と verifier は structured circuit なら $O(D \log C + \log^2 n)$。
- Aurora との最大の違いは、RS / FRI 型ではなく、multilinear / GKR 型であること。
- Virgo の強みは structured circuit と相性がよい点にある。

次章では、Virgo を小さな layered circuit で具体的に追い、GKR と zkVPD の役割分担を体感します。

Virgo を手で追う

この章では、Virgo の流れを小さな layered circuit で追います。
ここでも最初に断っておきますが、**以下は原論文の完全な transcript を再現するものではなく、GKR + zkVPD の役割分担を理解するための模式図**です。

6.1 例題となる layered circuit

次の 4 入力の回路を考えます。

$$z = (x_0 \cdot x_1) \cdot (x_2 + x_3)$$

具体的な値として

$$x_0 = 2, \quad x_1 = 3, \quad x_2 = 4, \quad x_3 = 1$$

を入れると、

- $a = x_0 x_1 = 6$
- $b = x_2 + x_3 = 5$
- $z = a \cdot b = 30$

です。

これを layer 化すると、

- layer 2: 入力 x_0, x_1, x_2, x_3
- layer 1: 中間ゲート a, b
- layer 0: 出力 z

という 3 層の回路になります。

6.2 Step 1: 各 layer の値を表にする

各 layer の gate 値を表にします。

6.2.1 入力 layer

$$V_2(00) = 2, \quad V_2(01) = 3, \quad V_2(10) = 4, \quad V_2(11) = 1$$

ここでは 2 ビット添字で 4 個の入力を表したと思えば十分です。

6.2.2 中間 layer

$$V_1(0) = 6, \qquad V_1(1) = 5$$

- gate 0 は乗算ゲートで x_0, x_1 を子に持つ
- gate 1 は加算ゲートで x_2, x_3 を子に持つ

6.2.3 出力 layer

$$V_0(0) = 30$$

このように、各 layer を「添字 $\mapsto$ 値」の表として持ちます。

6.3 Step 2: その表を multilinear extension に持ち上げる

GKR / Virgo は、これらの有限表をそのまま扱うのではなく、multilinear extension に持ち上げます。

- $\tilde{V}_2$: $\{0,1\}^2$ 上の値表を $\mathbb{F}^2$ 上へ拡張した多項式
- $\tilde{V}_1$: $\{0,1\}$ 上の値表を $\mathbb{F}$ 上へ拡張した多項式
- $\tilde{V}_0$: 出力の定数多項式に近いもの

直感的には、

「離散的な gate 値表を、ランダム点で評価できる低次数関数に変換する」

ということです。

これで verifier は、表全体を読む代わりに、ランダム点 r での $\tilde{V}_i(r)$ を主張として追えます。

6.4 Step 3: 検証したい主張を 1 個に圧縮する

本当に検証したいのは「出力が 30 で、しかも各ゲート計算が全部正しい」ことです。
しかし各ゲートを全部見るのは重い。

そこで GKR は、layer 全体の正しさを **1 個のランダム点評価の主張** に圧縮します。

この例だと出力 layer はほぼ 1 個の gate しかないので、

$$\tilde{V}_0(r_0) = 30$$

のような主張から始まると考えてよいです。実際には r_0 は trivial でも構いません。

重要なのはここから先です。
Virgo は「出力 gate が正しい」という主張を、「中間 layer の値表が正しい」という主張へ落としていきます。

6.5 Step 4: GKR が layer 0 の主張を layer 1 へ落とす

出力 gate は乗算ゲートで、

$$z = a \cdot b$$

でした。
したがって出力の正しさは、中間 layer の値 a, b の正しさに帰着できます。

しかし GKR は単に「a と b を送れ」とは言いません。
そうではなく、layer 1 の値表 V_1 全体に関する主張を、ある低次数多項式の和として組み立て、それを sumcheck で圧縮します。

6.5.1 何を sumcheck するのか

詳細な式を省いて直感だけ言うと、GKR は次のような情報を 1 本の多項式へまとめます。

- その gate が加算か乗算か
- どの子ゲートに接続しているか
- 子ゲートの値が何か
- それらから親ゲート値が正しく計算されるか

この「gate の配線と値の整合性」をまとめた多項式の総和を検査することで、layer 0 の主張を layer 1 のランダム点評価主張へ縮約します。

6.6 Step 5: sumcheck による圧縮のイメージ

layer 1 には 2 個の gate があります。
本当に正しさを確かめようとすると、

- gate 0 は乗算か
- gate 0 の子は正しいか
- gate 1 は加算か
- gate 1 の子は正しいか

などを全部見る必要があります。

GKR はこれを、多項式 g の和の検査

$$\sum_{b \in \{0,1\}^\ell} g(b)$$

へ落とします。
そして classical sumcheck により、その主張を最終的に **ランダム点での g の 1 回評価** へ圧縮します。

この圧縮の結果、検証者は「layer 1 全体の正しさ」を直接見るのではなく、

- $\tilde{V}_1(r_1)$
- ときには $\tilde{V}_1(r_1')$

のような少数の評価主張だけを追えばよくなります。

6.7 Step 6: 同じことを layer 1 から入力 layer へ繰り返す

中間 layer の 2 つの gate についても、正しさは入力 layer の値に依存します。

- $a = x_0 x_1$
- $b = x_2 + x_3$

ですから、layer 1 の主張も最終的には layer 2（入力 layer）の主張へ落ちます。

こうして GKR は、

出力 `layer` の主張
↓
中間 `layer` の主張
↓
入力 `layer` の主張

と縮約していきます。

6.7.1 これのどこがすごいのか

回路全体が大きくても、検証者は各 layer の gate を全列挙しません。
各段階で必要なのは、ランダム点での評価主張と、その整合性だけです。
これにより、巨大回路でも verifier の仕事がかなり小さくなります。

6.8 Step 7: では $\tilde{V}_i(r)$ は誰がどう計算するのか

ここで zkVPD が登場します。

GKR の縮約では「では $\tilde{V}_1(r_1)$ はいくつですか」「では $\tilde{V}_2(r_2)$ はいくつですか」という問いが何度も出ます。
しかし verifier は巨大な表を持ちたくありませんし、自分で多項式補間もしたくありません。

そこで prover が

$$y = \tilde{V}_i(r)$$

という値を提示し、その正しさを **短い証明**付きで示します。
これが VPD 的役割です。

6.9 Step 8: zkVPD の役割を直感で理解する

zkVPD は、Virgo の文脈では次のように理解するとよいです。

- prover は巨大な値表（またはそこから導かれる多項式）を内部に持っている
- verifier は、その多項式の特定点での値だけ知りたい
- しかも、その値が本当に内部表から導かれたものかを確かめたい
- さらに、その過程で表の中身を漏らしたくない

したがって zkVPD は、

> **巨大な表への完全アクセスを与えずに、必要な点評価だけを正しく・短く・秘匿的に委任する仕組み**

です。

6.9.1 GKR と zkVPD の役割分担

ここは重要なので、もう一度書きます。

- **GKR** は「正しさの論理」を作る
- **zkVPD** は「その論理で要求される点評価を短く安全に提供する」

Virgo を理解するとき、この分担を取り違えないことが最重要です。

6.10 Step 9: structured circuit が効く場所

Virgo の verifier が軽くなるためには、wiring predicate の評価が速くないといけません。この例では配線が非常に単純なので、verifier は

- gate 0 は入力 00,01 を掛ける
- gate 1 は入力 10,11 を足す

と簡単に理解できます。

大規模回路でも、配線が規則的なら verifier は「ランダムな gate 添字を与えられたとき、その子添字を高速に計算する」ことができます。これが structured circuit の旨味です。

逆に、配線が無秩序だと verifier は wiring 自体を重く読まねばならず、Virgo の理論上の succinctness がそのまま出にくくなります。

6.11 Step 10: 非対話化とゼロ知識化

Virgo も最終的には非対話 proof system として使いたいので、ハッシュ・Merkle・Fiat–Shamir 的な部品を使います。

また、GKR が要求する評価主張や zkVPD のやり取りは、そのままだと witness 情報をにじませる可能性があります。したがって、Virgo でもランダムマスキングを含む zero-knowledge 化が必要です。

ここでの直感は Aurora と同じです。

- 検証に必要な整合性は残す
- それ以外の自由度は乱数で埋める

ただし、Aurora では「低次数オラクル全体の世界」が主役だったのに対し、Virgo では「multilinear evaluations の世界」が主役です。

6.12 この例で verifier が本当に納得するもの

例題では、最終的に verifier は次を信じます。

- 入力表 V_2 と整合する何らかの計算が存在し
- その計算の layer 1 の値表が正しく
- 出力 layer の値が 30 である

しかも、その過程で prover は x_0, x_1, x_2, x_3 の生値や全中間値表をそのまま提出しません。
検証者は「少数のランダム点での正しい評価主張」と「それを支える短い証明」しか見ません。

6.13 Virgo の理解でつまずきやすい点

6.13.1 1. GKR が“表の全部を見ている”と思う

違います。
GKR の本質は、表全体の正しさを sumcheck で圧縮することにあります。

6.13.2 2. zkVPD が GKR を置き換えていると思う

違います。
zkVPD は GKR の代替ではありません。GKR が作った評価主張を外から扱いやすくする部品です。

6.13.3 3. verifier が常に速いと思う

structured circuit という条件が重要です。
配線の規則性がないと verifier の旨味は薄れます。

6.13.4 4. Virgo は単に“FRI の代わりに別の PCS を使っただけ”と思う

そうではありません。
Virgo は回路の見方からして Aurora とかなり違います。Aurora の中心が R1CS と RS codeword なら、Virgo の中心は layered circuit と GKR です。

6.14 Virgo を一行で再定義する

ここまでを一文にすると、Virgo は次のように言えます。

> **layered arithmetic circuit の正しさを GKR で layer-by-layer に縮約し、その過程で現れる多項式評価主張を transparent な zkVPD で短く証明する系**

この一文が腑に落ちれば、Virgo の設計思想はかなり掴めています。

6.15 この章のまとめ

- Virgo は layer ごとの値表を multilinear extension として見る。
- GKR は、上の layer の正しさを下の layer の主張へ段階的に縮約する。
- 各段階で現れる「多項式のランダム点評価」の主張を、zkVPD が短く安全に支える。
- structured circuit なら verifier は wiring を高速評価でき、理論上の succinctness が活きる。
- Aurora が RS code / FRI を前景に出すのに対し、Virgo は GKR / multilinear evaluations を前景に出す。

次章では、Aurora と Virgo を横に並べて、表現・複雑性・ユースケース・実装の観点から比較します。

Aurora と Virgo の比較

この章では、Aurora と Virgo を真正面から比較します。
両方とも transparent な零知識証明系ですが、「何を自然な入力表現とみなすか」「どこで計算を圧縮するか」「何が verifier の律速になるか」がかなり違います。

7.1 まずは早見表

項目	Aurora	Virgo
主対象	R1CS	layered arithmetic circuit
基本的な多項式表現	Reed–Solomon codeword / univariate	multilinear extension / Boolean hypercube
中核技法	Encoded IOP + univariate sumcheck + FRI	GKR + zkVPD
trusted setup	不要	不要
暗号的前提	ハッシュ中心、lightweight cryptography	ハッシュ中心、lightweight cryptography
証明サイズ	$O(\log^2 n)$（制約数 n）	$O(D \log C + \log^2 n)$
証明生成	$O(n \log n)$	$O(C + n \log n)$
検証	$O(n)$	structured circuit なら $O(D \log C + \log^2 n)$
直感的な強み	transparent なのに短い proof size	structured circuit で軽い verifier
読み解く鍵	低次数性と FRI	layer 縮約と GKR

この表は出発点にすぎません。以下で、「なぜそう違うのか」を見ます。

7.2 比較 1: 入力表現の違い

7.2.1 Aurora は R1CS が自然言語

Aurora は R1CS を直接の対象とします。
したがって、

- すでに R1CS へ落とすコンパイラがある
- 回路や命題を一様な rank-1 constraint へ揃えたい
- 低次数コード語ベースの設計へ乗せたい

という状況で自然です。

Aurora の発想では、まず「制約系をどう低次数多項式の整合性へ埋め込むか」が中心課題

になります。

7.2.2 Virgo は layered circuit が自然言語

Virgo は layered arithmetic circuit を重視します。
したがって、

- 計算に自然な層構造がある
- wiring が規則的
- GKR の layer-by-layer 縮約が効く

という状況で自然です。

Virgo の発想では、まず「回路をどう層ごとの値表へ分けるか」が中心課題になります。

7.2.3 どちらが“より一般”か

この問いには注意が必要です。
R1CS と回路は多項式時間で相互変換できるので、**理論的表現力だけを見れば大きな差はありません**。
しかし実務上は、「変換後にどの構造が残るか」が重要です。

- R1CS 化すると一様性が得られる
- layer 化すると構造的規則性が見やすい

つまり、表現力の差というより **どの構造を活かす設計か** の差です。

7.3 比較 2: 多項式を見る幾何学の違い

この比較が最重要です。

7.3.1 Aurora は univariate / evaluation domain の幾何学

Aurora は大きな表を

- 評価領域 H 上の
- 低次数 univariate 多項式の評価表

として見ます。

したがってキーワードは

- Reed–Solomon code
- low-degree test
- FRI
- codeword consistency

です。

7.3.2 Virgo は multilinear / hypercube の幾何学

Virgo は layer ごとの値表を

- $\{0,1\}^\ell$ 上の関数
- の multilinear extension

として見ます。

したがってキーワードは

- Boolean hypercube
- multilinear extension
- sumcheck
- wiring predicate
- GKR

です。

7.3.3 教訓

Aurora と Virgo の違いは、「FRI を使うか GKR を使うか」よりもっと深いところにあります。
それは **“巨大な計算表を、どういう多項式として眺めるか”**の違いです。

この違いを理解すると、後続の多くの zk 系列も整理しやすくなります。

7.4 比較 3: 何を圧縮しているのか

7.4.1 Aurora が圧縮しているもの

Aurora が圧縮しているのは、

- 大量の R1CS 制約
- オラクル表全体の低次数性
- それらの整合性

です。

圧縮の主役は、IOP と FRI、そして univariate sumcheck 的なチェックです。

7.4.2 Virgo が圧縮しているもの

Virgo が圧縮しているのは、

- 回路 layer 間の整合性
- 各 layer の主張を次 layer へ落とす過程
- 途中で出てくる多項式評価主張

です。

圧縮の主役は GKR であり、zkVPD はその圧縮結果を短く外部化する部品です。

7.5 比較 4: verifier の性格

7.5.1 Aurora verifier

Aurora の verifier は $O(n)$ です。
ここで n は制約数です。

これは proof size が非常に短いことと対照的です。
Aurora は「verification が極小」よりも、「transparent で短い証明」の側へ重心があります。

7.5.2 Virgo verifier

Virgo の verifier は structured circuit なら

$$O(D \log C + \log^2 n)$$

です。

したがって、回路が規則的であれば、Aurora より verifier 観点で有利に見えることがあります。

7.5.3 実務的な読み方

本書の整理としては、

- **verification cost を最重要視**し、回路が structured なら Virgo が魅力的
- **proof size の短さと transparent 性の組合せ**を重視するなら Aurora が魅力的

という見方ができます。

ただし、実装・エコシステム・具体定数は別問題なので、複雑性式だけで最終判断してはいけません。

7.6 比較 5: prover の性格

7.6.1 Aurora prover

Aurora の prover は $O(n \log n)$。
低次数コード語への変換や FRI 的処理が中心にあります。

7.6.2 Virgo prover

Virgo の prover は $O(C + n \log n)$。
layer ごとの処理が回路サイズ C に素直に乗り、さらに入力符号化のコストが付くイメージです。

7.6.3 どう解釈するか

Aurora も Virgo も“重い”ことには変わりません。
現代の zk 証明系一般がそうですが、prover はだいたい最大の負担を背負います。

違いは、その負担が

- Aurora では「低次数コード語と FRI の世界」で現れるか
- Virgo では「GKR と評価委任の世界」で現れるか

です。

7.7 比較 6: 実装難度の違い

7.7.1 Aurora の実装難所

- R1CS からの多項式的翻訳
- 多数のオラクル管理

- FRI のパラメータ管理
- Merkle / transcript の整合性
- ゼロ知識マスキング

Aurora の難しさは、「低次数オラクルの大規模な管理」にあります。

7.7.2 Virgo の実装難所

- 回路の layer 化
- wiring predicate の efficient 実装
- GKR の各ラウンド
- multilinear evaluations
- zkVPD の設計・実装

Virgo の難しさは、「回路構造と評価主張をどう効率良く扱うか」にあります。

7.7.3 どちらが難しいか

単純な優劣はありません。
Aurora は符号理論寄りの難しさがあり、Virgo は GKR / multilinear extension / wiring 寄りの難しさがあります。
研究者がどちらを“自然”と感じるかは、背景知識にかなり依存します。

7.8 比較 7: どんな回路に向くか

7.8.1 Aurora が向きやすい回路

- すでに R1CS コンパイラが整っている
- 一般的な制約記述を素直に受け取りたい
- 低次数検査ベースで設計したい

7.8.2 Virgo が向きやすい回路

- 明確な layer 構造がある
- 配線が規則的
- verifier も軽くしたい
- Merkle 木や線形代数のような反復構造がある

7.8.3 重要な補足

ここでいう「向く」は、**本書の設計的整理**です。
原論文がすべての応用で一方の優位性を実験的に証明している、という意味ではありません。
実際の選択には、ライブラリ、最適化、ハードウェア、周辺 gadget など多くの要素が効きます。

7.9 Aurora と Virgo は競合か、補完か

私は、**競合であり補完でもある**と考えるのが自然だと思います。

- 競合: どちらも transparent zk proof を目指している
- 補完: 設計思想が大きく異なるため、適した計算構造が違う

Aurora を理解すると、FRI / IOP / RS code 系の透明証明を読む力が上がります。
Virgo を理解すると、GKR / multilinear / evaluation-delegation 系の証明を読む力が上がります。
両方を学ぶ意味は、まさにこの補完性にあります。

7.10 どちらを選ぶべきか：簡易意思決定表

7.10.1 Aurora を第一候補にしやすいとき

- 既存パイプラインが R1CS ベース
- transparent setup が欲しい
- proof size をかなり短くしたい
- FRI / low-degree testing を主役にしたい

7.10.2 Virgo を第一候補にしやすいとき

- 回路が layered で structured
- verifier をできるだけ軽くしたい
- GKR 的な縮約が効く
- 規則的な wiring を活かせる

7.10.3 どちらでもなく、他系も見るべきとき

- エコシステム重視で Plonkish 系を使いたい
- trusted setup を許容してでも極短 proof を優先したい
- 既存実装・監査済みライブラリを最優先したい
- 再帰合成や特定 VM 向け最適化を主目標にしたい

この最後の項目は、本書の比較を超えて現代的選定の話になります。

7.11 研究者にとっての学習価値

Aurora と Virgo を両方学ぶと、少なくとも次の 2 本の設計線が見えるようになります。

1. **RS code / low-degree test / FRI 線**
2. **hypercube / multilinear extension / GKR 線**

この 2 線は、近年の多くの zk 証明系の祖型になっています。
したがって、Aurora と Virgo を読むことは、単に 2 本の論文を知ることではなく、**透明証明系の地図を得る**ことに近いです。

7.12 この章のまとめ

- Aurora と Virgo は、どちらも transparent だが、入力表現と圧縮の思想が大きく異なる。
- Aurora は R1CS と univariate / RS code / FRI の世界に立つ。
- Virgo は layered circuit と multilinear / GKR / zkVPD の世界に立つ。
- Aurora は proof size の短さが印象的で、Virgo は structured circuit での軽い verifier が魅力的である。
- どちらが適切かは、表現力の優劣ではなく、**どの構造を活かしたいか**で決まる。

次章では、これらの違いを具体的なユースケースへ落とし込み、「どんな問題にどう考えるか」を見ます。

ユースケースと選び方

この章では、Aurora と Virgo がどのような問題に自然に現れるかを、具体例で考えます。
ただし最初に重要な注意を述べます。

8.1 注意：この章の「ユースケース」は設計上の適合例である

この章で述べる多くの例は、

- 「原論文でそのまま本番採用された」と主張するもの
- 「この系が常に最速・最善である」と断定するもの

ではありません。

むしろ、

Aurora / Virgo の設計思想が、どういう問題構造と相性がよいか

を理解するための **概念上の適合例** です。

したがって、この章は「文献上の定理」よりも「設計上の判断」に近い章です。研究・実装では、必ず具体ライブラリ、定数因子、メモリ、再帰性、監査状況も合わせて見てください。

8.2 ユースケース 1: Merkle 木の証明

Merkle 木は、Virgo を理解する上で非常に典型的です。
実際、Virgo 原論文でも 256 leaves の Merkle tree 回路が評価対象として使われています。[Virgo20]

8.2.1 問題設定

たとえば、あるデータ項目 m が Merkle root R に含まれていることを証明したいとします。

公開情報:

- root R
- leaf index i

秘密情報:

- leaf 値 m
- authentication path

検証したいのは、「path を順にハッシュすると root R になる」という計算です。

8.2.2 なぜ Virgo と相性がよいのか

Merkle 木は

- 層構造が明確

- 各層で同じようなハッシュ計算を繰り返す
- wiring が規則的

なので、layered circuit として非常に自然です。
したがって GKR 的縮約と相性がよく、Virgo の設計感覚がよく出ます。

8.2.3 Aurora ではどうか

Aurora でももちろん可能です。
Merkle 計算を R1CS に落とせばよいからです。
したがって、

- 既存の R1CS コンパイラを使いたい
- Aurora 系の low-degree / FRI 線で統一したい

なら Aurora 的な発想も自然です。

8.2.4 まとめ

- **Merkle 木 = Virgo が分かりやすく得意そうに見える典型例**
- ただし Aurora でも表現できる

8.3 ユースケース 2: バッチ検証・ロールアップ的計算

多数の取引や操作を 1 本の回路へまとめ、その正しさを 1 本の zk proof で示したい場面を考えます。

8.3.1 問題設定

- 多数のトランザクション更新
- 各更新で署名検査や残高更新
- 最後に新しい状態 root を出力
- その全体が正しいことを 1 本の証明で示す

これは zk-rollup 的発想の典型です。

8.3.2 Aurora が自然に見える状況

- 既存ツールチェーンが R1CS ベース
- 署名検査や状態更新を一様な制約列として作りやすい
- transparent setup を強く重視
- proof size もかなり気にする

このとき Aurora は非常に自然です。
回路の不規則性がある程度あっても、R1CS に落としてから低次数的に扱う発想に乗せやすいからです。

8.3.3 Virgo が自然に見える状況

一方で、

- 更新規則がかなり規則的
- 同型の処理が多数並ぶ
- 状態木や反復演算が layered に見やすい

なら Virgo 的発想も魅力的です。
ただし、rollup の回路は現実には range check、署名、bit decomposition、lookup 的処理などが複雑に絡むので、「理論的に Virgo がよさそう」だけで即決はできません。

8.3.4 まとめ

- 不規則な一般制約列として扱うなら Aurora は見通しがよい
- 規則的な反復構造が強いなら Virgo も候補に入る

8.4 ユースケース 3: プライバシー保護された資格証明・KYC

たとえば、利用者が次を証明したいとします。

- 年齢が 20 歳以上
- 特定国籍ではない
- 制裁リストに載っていない
- ただし氏名や生年月日は見せたくない

8.4.1 証明したい計算

この種の問題は、結局は

- ハッシュ整合性
- 署名検証
- 比較・範囲判定
- 属性値の選択的開示

といった算術・論理条件になります。

8.4.2 Aurora 的に見ると

属性や制約を R1CS に落とし、全体を 1 本の一般制約系として扱うのはかなり自然です。
とくに「既に回路コンパイラで一般命題を R1CS に落とせる」なら、Aurora 的発想はまっすぐです。

8.4.3 Virgo 的に見ると

属性証明の回路は、Merkle path やデータ並列処理を含むなら Virgo 的構造が現れます。
ただし、不等式・範囲証明・ビット制約・非線形 gadget が多い場合は、structured さが崩れやすいので、Virgo の旨味は状況依存になります。

8.4.4 設計メモ

資格証明系では、プロトコル選択以上に

- range check のコスト
- ブール化のコスト
- ハッシュ関数の算術化コスト
- フィールド選択

が支配的になることが多いです。これは Aurora / Virgo のどちらでも避けられません。

8.5 ユースケース 4: 行列演算・機械学習・線形代数

機械学習推論や線形代数演算は、Virgo 的な視点と相性がよいことが多いです。

8.5.1 例

- 行列積
- 畳み込み
- 線形層
- データ並列な反復演算

これらはしばしば

- 層構造が明快
- 配線規則が単純
- 同じ計算パターンが大量反復

という特徴を持ちます。

8.5.2 なぜ Virgo 的か

GKR は、こうした規則的な回路を layer ごとに圧縮するのが得意です。
したがって、「計算全体を一般制約列に潰してから見る」より、「もともとの層構造を活かして見る」方が自然な場合があります。

8.5.3 ただし注意

機械学習回路では、しばしば次が問題になります。

- ReLU や比較演算の近似
- 浮動小数点を有限体へ埋め込む誤差
- 大量の範囲制約
- 非 native な演算

したがって、「線形代数だから即 Virgo」とは言えません。
しかし **規則性の強い層計算をどう扱うか** という観点では、Virgo 的発想は非常に教育的です。

8.6 ユースケース 5: クラウドへの計算委託(verifiable outsourcing)

依頼者が重い計算をサーバへ渡し、その出力の正しさだけを速く確認したい、というのは verifiable computation の古典的な動機です。

8.6.1 Aurora 的に自然な見方

- 任意計算を回路 / R1CS 化
- 証明者が長い証明を作る
- 検証者は短い proof を確認する

Aurora は transparent であることもあり、「誰が設定を作ったか」に依存しない形で委託検証したい場面と相性がよいです。

8.6.2 Virgo 的に自然な見方

- 計算が layered かつ規則的
- verifier 側をできるだけ軽くしたい

この場合は Virgo 的な縮約が魅力的です。
GKR はもともと verifiable computation の文脈で非常に重要な技法です。

8.7 ユースケース 6: 再利用される固定回路

たとえば「同じ計算テンプレートを何度も違う入力で証明する」場面を考えます。

- 同じ認証回路
- 同じバッチ検証回路
- 同じ Merkle 更新回路

このとき、回路の構造が固定されるので、Virgo では structured 性を活かしやすい場合があります。一方で Aurora でも、R1CS 化された制約系が固定なら開発フローを整理しやすい利点があります。

8.7.1 何を見るべきか

こうしたケースでは、「理論式」だけでなく次を見ます。

- 回路生成のしやすさ
- 実装メモリ
- オラクル / transcript の扱いやすさ
- ライブラリの成熟度
- テストのしやすさ

つまり、同じ固定回路でも、Aurora 向きか Virgo 向きかはエンジニアリング次第です。

8.8 逆に、どんなときに相性が悪いか

8.8.1 Aurora が相対的に苦しく見えやすいとき

- verifier をとにかく小さくしたい
- structured circuit が強く効く
- R1CS へ潰すことで本来の規則性が見えにくくなる

8.8.2 Virgo が相対的に苦しく見えやすいとき

- 回路が非構造的
- wiring の規則性が乏しい
- range checks や bit gadget が多く structured 性が崩れる
- 元から R1CS 工具立てで全部組まれている

ここでも「相対的に」という言い方をしている点が大事です。
現代の実装では定数因子や最適化で印象が大きく変わります。

8.9 実践的な選び方のフローチャート

8.9.1 質問 1: 入力は自然に R1CS になるか、layered circuit になるか

- R1CS が自然 → Aurora 寄り
- layered / structured が自然 → Virgo 寄り

8.9.2 質問 2: verifier の軽さはどれくらい重要か

- proof size を短くしたいが verifier 線形でも許容 → Aurora あり
- verifier をかなり軽くしたい → Virgo を強く検討

8.9.3 質問 3: 回路はどれだけ規則的か

- 高い規則性 → Virgo の旨味が出やすい
- 不規則・gadget 多め → Aurora 的整理の方が扱いやすいことが多い

8.9.4 質問 4: 実装上の制約は何か

- ハッシュ / FRI 系の知見があるか
- GKR / multilinear evaluation 系の知見があるか
- 既存コード資産はどちら向きか

最後はこの質問が効きます。
理論だけでなく、**誰がどの技術を理解しているか** も大きな選定要因です。

8.10 この章のまとめ

- Merkle 木や規則的な層計算では Virgo 的発想が自然に見えやすい。
- 一般的な制約列や R1CS 工具立てがある状況では Aurora 的発想が自然に見えやすい。
- バッチ検証、資格証明、ML、委託計算など、多くの応用はどちらでも表現できる。
- 重要なのは「表現できるか」ではなく、「どの構造を活かして証明系を組むか」である。
- 本章のユースケースは、文献上の運用実績の主張ではなく、設計上の適合例として読むべきである。

次章では、実装・研究・セキュリティの観点から、Aurora / Virgo を使うときに注意すべき点を整理します。

実装・研究・セキュリティ上の注意点

この章は、Aurora / Virgo を「論文として理解する」段階から一歩進めて、**もし研究実装するなら何に気をつけるか**を整理する章です。
ここでは、文献に直接基づく事実と、本書としての設計上の助言をできるだけ分けて書きます。

9.1 まず最初に：論文理解と本番採用は別物

これは最重要のメッセージです。

- ある系が理論的に美しい
- 論文が歴史的に重要
- 教育的価値が高い

ことと、

- 今日そのまま本番実装に使うべき
- 監査済みで枯れている
- 周辺エコシステムが整っている

ことは、まったく同じではありません。

Aurora と Virgo は、いずれも設計原理を学ぶ上で非常に重要です。しかし実装選定では、後続研究・セキュリティ分析・コード品質・運用要件を別途必ず確認してください。

9.2 文献上の事実 1: Virgo は後続研究で再検討されている

2025 年の後続研究 *Revisiting Virgo: A Study of Vulnerabilities, Limitations, and Optimizations* は、Virgo の zkVPD 設計と実装について、脆弱性・制約・見落とされていたコストを指摘しています。[VirgoRevisit25]

この論文が主張する要点を、本書の関心に即して要約すると次の通りです。

- Virgo の zkVPD 設計・実装に exploitable な security issue があった
- 一部のマスキング手続きが実装で欠落していた
- 実用上のフィールド選択に制約があり、range check の負担が過小評価されていた
- binary field やその拡大体への対応には難しさがあった
- 改良版 VPD / zkVPD が提案された

9.2.1 どう読むべきか

したがって、Virgo は

- **歴史的・教育的には極めて重要**
- しかし **そのまま素朴に実装すれば安全とは限らない**

と理解するのが適切です。

この区別は非常に大切です。
特に研究実装では、「論文を読んでそのまま書いたコード」が安全とは限りません。

9.3 文献上の事実 2: FRI の Fiat–Shamir 安全性も後続研究で精密化された

FRI を含む非対話 proof system の安全性、特に Fiat–Shamir 変換後の安全性は、後続研究でさらに精密に解析されています。[FSFRI23]

この事実から得る教訓は単純です。

> 「transparent で hash-based だから自動的に全部安心」と考えてはいけない

ということです。

Aurora のように FRI を使う系では、

- どの transcript をどうハッシュするか
- challenge の導出をどう domain separation するか
- どの順序で message を固定するか

といった実装細部が非常に重要です。

9.4 文献上の事実 3: “lightweight cryptography”は“実装が簡単”を意味しない

Aurora も Virgo も、pairing のような重い公開鍵部品ではなく、主に

- ハッシュ関数
- 有限体演算
- 低次数テスト / 和検査
- オラクル管理

で構成されます。

これはしばしば“lightweight cryptography”と呼ばれます。
しかしこれは「数学的仮定が軽い」「透明性が高い」という意味であって、**エンジニアリングが簡単**という意味ではありません。

実際には、

- Merkle 証明の大量処理
- transcript 管理
- 大規模配列のメモリアクセス
- フィールド演算最適化
- 乱数・マスキングの実装

など、非常に多くの実装課題があります。

9.5 設計上の助言 1: まず“回路の自然な形”を見よ

ここからは本書の助言です。

Aurora と Virgo を選ぶとき、最初に見るべきなのは複雑性式ではなく、**対象計算の自然な形**です。

9.5.1 Aurora 寄りのサイン

- 一般命題を R1CS に落とすのが自然
- 既存パイプラインが constraint system 中心
- 回路が不規則
- low-degree / FRI 系の知見が手元にある

9.5.2 Virgo 寄りのサイン

- layer 構造が明確
- wiring が規則的
- 同型パターンが反復
- verifier をかなり小さくしたい

複雑性式は、その後で見れば十分です。

9.6 設計上の助言 2: フィールド選択は“最後の微調整”ではない

有限体の選択は、実装の最初に決めるべき事項です。

なぜなら、フィールド選択は

- 使いやすい FFT / coset 構造
- ハッシュ算術化のコスト
- range check の必要性
- non-native arithmetic の有無
- 具体セキュリティ

に全部効くからです。

特に Virgo の後続研究が示すように、「特定の都合のよいフィールド」を前提にすると、後で別の演算を表現しにくくなったり、range check の爆発が起きたりします。[VirgoRevisit25]

9.7 設計上の助言 3: range check と bit decomposition を甘く見ない

これは多くの zk 実装で本当に重要です。

- 比較
- 範囲制約
- ブール値制約
- ハッシュ入出力の bit 化
- 小さい整数の埋め込み

こうした処理は、しばしば本体計算より重くなります。

Aurora / Virgo の選択以前に、

> 「本当に証明したい命題の大半が、実は range check ではないか」

を確認すべきです。

特に「本来は小さな整数演算なのに、大きな有限体上に持ち上げている」場合、このギャップが実装コストを決めます。

9.8 設計上の助言 4: メモリを計算時間と同じくらい重視する

論文を読むと prover time に目が行きがちですが、実装ではメモリがしばしば先に詰まります。

9.8.1 Aurora 側で起きやすいこと

- 大きなオラクル表を複数持つ
- FRI の各段階で中間表を持つ
- Merkle 木の葉・内部ノードを管理する

9.8.2 Virgo 側で起きやすいこと

- layer ごとの値表を持つ
- multilinear evaluation のためのデータ構造を持つ
- zkVPD のコミットメント用に大きな表を保持する

したがって、理論上の演算回数だけでなく、

- peak memory
- stream 処理の可否
- GPU / SIMD への載せやすさ
- cache-friendly かどうか

も初期設計から考えるべきです。

9.9 設計上の助言 5: transcript の domain separation を徹底する

これは暗号プロトコル実装の常識ですが、特に IOP / FRI / Fiat–Shamir 系では致命的に重要です。

最低限、

- ラウンド番号
- オラクル種別
- root hash
- 開示位置
- パラメータ集合
- インスタンス識別子

を十分に区別してハッシュすべきです。

「適当に全部連結してハッシュ」は危険です。
研究実装であっても、ここは甘くしない方がよいです。

9.10 設計上の助言 6: “structured”を曖昧語のまま使わない

Virgo の議論で最も危険な曖昧語は structured です。
実装計画を書くなら、必ず次を明記してください。

- gate 添字はどう表現するか
- wiring predicate はどの時間で計算できるか
- random point での wiring 評価はどう実装するか

- 層ごとの fan-in / fan-out はどうなっているか

これを曖昧にしたまま「Virgo 向きです」と言うのは危険です。

9.11 設計上の助言 7: Aurora でも Virgo でも、ゼロ知識の確認は別枠で行う

多くの実装バグは健全性よりも **ゼロ知識性の破れ** として現れます。

- 本来入れるべき blinding が入っていない
- 一部オラクルだけマスクされていない
- transcript から内部値が相関的に漏れる
- テストケースが“小さすぎて”漏洩に気づけない

Virgo の後続研究が示す通り、ここは本当に見落としやすいです。[VirgoRevisit25] したがって、

- 形式的に何を隠すべきか
- どの message に乱数が必要か
- 同じインスタンスを複数回証明したときの相関はどうか

を別枠でレビューしてください。

9.12 実験を書くときの注意

研究論文や卒論・修論で実装評価を書くなら、最低限次を分けて報告するとよいです。

1. **statement size**
2. **witness size**
3. **gate / constraint 数**
4. **prover time**
5. **verifier time**
6. **proof size**
7. **peak memory**
8. **field / hash choice**
9. **range check を含むかどうか**
10. **回路生成時間を含むかどうか**

このうち 8〜10 を省くと、比較がかなり不公平になります。

9.13 研究テーマとして何が面白いか

Aurora / Virgo 周辺で今でも面白いテーマは多いです。

- FRI 系の concrete security
- IOP からの効率的コンパイル
- multilinear evaluation の高速化
- structured circuit の自動抽出
- binary field への自然な対応
- マスキングと zero-knowledge の実装検証
- 分散 prover
- recursion との相性

つまり、Aurora と Virgo は“過去の完成品”というより、**今も研究線が伸びている祖型**だと見る方が実態に近いです。

9.14 実務的なまとめ

最後に、本書としての実務的な助言を短くまとめます。

- Aurora を学ぶ目的は、FRI / low-degree / transparent proof の設計線を理解すること。
- Virgo を学ぶ目的は、GKR / multilinear / evaluation delegation の設計線を理解すること。
- 本番採用では、必ず後続研究・監査・実装成熟度を見ること。
- とくに Virgo については、2025 年の再検討論文を読まずに「そのまま実装」は避けるべき。
- プロトコル選択では、表現力よりも“計算構造との相性”を重視すべき。

9.15 この章のまとめ

- 論文理解と本番採用は分けて考えるべきである。
- Virgo には後続研究で指摘された脆弱性・制約があり、そのままの実装採用は危険である。
- FRI 系でも Fiat–Shamir 後の安全性や transcript 設計を軽視してはいけない。
- Aurora / Virgo の選択では、回路の自然な形、フィールド選択、range check、メモリ、structured 性を先に見るべきである。
- ゼロ知識性の検証は、健全性とは別に厳密にレビューすべきである。

次章では、理解を定着させるための演習問題を用意します。

演習問題

この章では、本書の理解を定着させるための演習問題を並べます。
解答をすべて詳細に書くと本書の流れが重くなるため、ここでは **考えるための方向性** と **短いヒント** を付けます。

10.1 基礎確認

10.1.1 問題 1

ゼロ知識証明の 3 性質、完全性・健全性・ゼロ知識性を、それぞれ 2〜3 行で説明してください。
また、どれが欠けると何が困るかも述べてください。

ヒント

- 完全性 = 正しい人が通る
- 健全性 = 嘘つきが通れない
- ゼロ知識性 = 通ること以外を学ばせない

10.1.2 問題 2

「計算の正しさの証明」を relation $R(x, w) = 1$ の形で表現してください。
ここで x と w に何を入れるのかを、あなた自身の言葉で明示してください。

ヒント

- x は公開入力と公開出力
- w は秘密入力と中間値列
- R は回路制約を全部満たすこと

10.2 回路と R1CS

10.2.1 問題 3

次の式を算術回路に分解してください。

$$t = (a + b)(c + 5)$$

加算ゲートと乗算ゲートを明示し、中間変数を導入してください。

ヒント

まず $u = a + b$、$v = c + 5$、$t = u \cdot v$ と分けると簡単です。

10.2.2 問題 4

問題 3 の回路を R1CS にしてください。
変数ベクトルを自分で定め、各制約を

$$\langle A_i, z\rangle \cdot \langle B_i, z\rangle = \langle C_i, z\rangle$$

の形で書いてください。

ヒント

加算ゲートは

$$(u) \cdot 1 = a + b$$

のように書けます。
定数 1 を変数ベクトルの先頭に入れるのを忘れないでください。

10.2.3 問題 5

なぜ R1CS が Aurora にとって自然な入力表現なのか、150〜300 字で説明してください。

ヒント

「制約が全部同じ型に揃うこと」と「多項式恒等式へ翻訳しやすいこと」を押さえるとよいです。

10.3 多項式と低次数性

10.3.1 問題 6

次数 d の多項式が“剛性”を持つとはどういうことか、補間の観点から説明してください。
また、それがなぜランダム検査と相性がよいのかを述べてください。

ヒント

異なる 2 つの次数 d 多項式は、多くても d 点までしか一致できません。

10.3.2 問題 7

$\mathbb{F}_{17}$ 上で $f(X) = 3X + 2$ を $H = \{1, 2, 3, 4\}$ 上で評価し、Reed–Solomon 的な評価表を書いてください。
さらに、1 点だけ改ざんした表が「同じ 1 次多項式の評価表」でないことを説明してください。

ヒント

評価値は手計算で出せます。
「1 次多項式は 2 点で決まる」という事実を使うと説明しやすいです。

10.4 sumcheck と GKR

10.4.1 問題 8

classical sumcheck の 2 変数版を、自分の言葉で説明してください。
なぜ

$$\sum_{b_1,b_2\in\{0,1\}} g(b_1,b_2)$$

という主張が、最終的にランダム点での 1 回の評価主張へ落ちるのかを整理してください。

ヒント

- 1 ラウンド目: $s_1(X) = g(X,0) + g(X,1)$
- 2 ラウンド目: $s_2(X) = g(r_1, X)$

10.4.2 問題 9

GKR はなぜ layered circuit を好むのでしょうか。
「layer i の主張を layer $i+1$ の主張へ落とす」という表現の意味を説明してください。

ヒント

各ゲートの正しさは、1 つ下の layer の子ゲート値で決まります。
この局所性を layer 全体に拡張したものが GKR です。

10.4.3 問題 10

“structured circuit”とは何かを、wiring predicate という語を使って説明してください。
そのうえで、Merkle 木や行列演算が structured と見なされやすい理由を述べてください。

ヒント

「ランダムな gate 添字を与えられたとき、その接続先を速く計算できるか」が鍵です。

10.5 Aurora / Virgo の理解確認

10.5.1 問題 11

Aurora において、FRI は何を担当し、何を担当しないでしょうか。
100〜250 字で説明してください。

ヒント

FRI は主に低次数性の検査です。
制約そのものの algebraic encoding は別にあります。

10.5.2 問題 12

Virgo において、GKR と zkVPD の役割を比較してください。
「正しさの論理」と「点評価の委任」という語を使って説明するとよいです。

ヒント

- GKR = 証明の骨格
- zkVPD = 途中の評価主張を短く支える部品

10.5.3 問題 13

Aurora と Virgo を比較して、次のどちらにどちらが自然に見えるかを考えてください。

1. 不規則な一般回路
2. 規則的な Merkle 木回路

それぞれ理由も述べてください。

ヒント

- 一般回路 → R1CS 的一様化
- 規則的回路 → layered / structured の活用

10.6 実装と研究の視点

10.6.1 問題 14

なぜ「field choice は最後の微調整ではない」と言えるのでしょうか。
range check、non-native arithmetic、ハッシュ算術化の少なくとも 2 つに触れて説明してください。

ヒント

有限体の選び方は、表現できる演算とそのコストを大きく左右します。

10.6.2 問題 15

なぜ peak memory を prover time と同じくらい重視すべきなのでしょうか。
Aurora と Virgo それぞれについて、メモリが増えそうな要因を 1 つずつ挙げてください。

ヒント

Aurora では多数のオラクル / FRI 中間表、Virgo では layer 値表や multilinear evaluation 用データ構造が候補です。

10.6.3 問題 16

Virgo の後続研究で何が再検討されたか、本書の範囲で説明してください。
その事実から、研究実装者が得るべき教訓は何でしょうか。

ヒント

キーワードは - zkVPD の脆弱性 - マスキングの欠落 - フィールド制約 です。

10.7 発展問題

10.7.1 問題 17

Aurora と Virgo の違いを、次の空欄を埋める形で 1 文にまとめてください。

- Aurora は _____ の世界で計算を圧縮する。
- Virgo は _____ の世界で計算を圧縮する。

ヒント

Aurora は RS code / univariate / FRI、Virgo は multilinear / GKR / hypercube です。

10.7.2 問題 18

あなたが新しい透明 zk proof system を設計するとしたら、Aurora から何を借り、Virgo から何を借りますか。
300〜600 字で、1 つの設計案を書いてください。

ヒント

正解は 1 つではありません。
重要なのは、- どの表現を使うか - どこで圧縮するか - 低次数性や評価主張をどう扱うか を明示することです。

10.8 学習の目安

次の問いに即答できるようになれば、本書の到達目標はかなり達成できています。

- なぜ低次数多項式が zk 証明で重要なのか。
- Aurora で FRI は何をしているのか。
- Virgo で GKR は何をしているのか。
- R1CS と layered circuit はどう違うのか。
- structured circuit とは何か。
- Aurora と Virgo のどちらを、どんな回路に当てたくなるか。

演習は、手を動かして初めて意味があります。
式を実際に書き、回路を実際に layer 化し、sumcheck を 2 変数で追ってみてください。理解の質が一段上がります。

参考文献

この章では、本書で中心的に参照した文献を挙げます。
とくに **[Aurora19]** と **[Virgo20]** が本書の主文献です。

最重要文献

[Aurora19]

Eli Ben-Sasson, Alessandro Chiesa, Michael Riabzev, Nicholas Spooner, Madars Virza, Nicholas P. Ward,
Aurora: Transparent Succinct Arguments for R1CS,
Advances in Cryptology –EUROCRYPT 2019, pp. 103–128, 2019.
DOI: `10.1007/978-3-030-17653-2_4`
- Springer: https://doi.org/10.1007/978-3-030-17653-2_4 - ePrint: https://eprint.iacr.org/2018/828

本書での役割

Aurora の定義、複雑性、設計思想、univariate sumcheck 的部品、FRI ベース透明証明としての位置づけ。

[Virgo20]

Jiaheng Zhang, Tiancheng Xie, Yupeng Zhang, Dawn Song,
Transparent Polynomial Delegation and Its Applications to Zero Knowledge Proof,
2020 IEEE Symposium on Security and Privacy (SP), pp. 859–876, 2020.
DOI: `10.1109/SP40000.2020.00052`
- DOI: https://doi.org/10.1109/SP40000.2020.00052 - ePrint: https://eprint.iacr.org/2019/1482

本書での役割

Virgo の定義、複雑性、GKR + zkVPD の骨格、structured circuit 条件、実験値。

背景文献

[IOP16]

Eli Ben-Sasson, Alessandro Chiesa, Nicholas Spooner,
Interactive Oracle Proofs,
Theory of Cryptography (TCC 2016-B), pp. 31–60, 2016.
DOI: `10.1007/978-3-662-53644-5_2`

- DOI: https://doi.org/10.1007/978-3-662-53644-5_2 - ePrint: https://eprint.iacr.org/2016/116

本書での役割

IOP というモデル自体の背景。Aurora を読むときの基礎。

[FRI18]

Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, Michael Riabzev,
Fast Reed–Solomon Interactive Oracle Proofs of Proximity,
ICALP 2018, LIPIcs 107, 14:1–14:17, 2018.
DOI: `10.4230/LIPIcs.ICALP.2018.14`
- DOI: https://doi.org/10.4230/LIPIcs.ICALP.2018.14 - Dagstuhl: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ICALP.2018.14

本書での役割

FRI の背景。Aurora の低次数性検査を理解するための基礎文献。

[GKR08]

Shafi Goldwasser, Yael Tauman Kalai, Guy N. Rothblum,
Delegating Computation: Interactive Proofs for Muggles,
STOC 2008, pp. 113–122, 2008.
DOI: `10.1145/1374376.1374396`
- DOI: https://doi.org/10.1145/1374376.1374396 - Microsoft Research copy: https://www.microsoft.com/en-us/research/wp-content/uploads/2016/12/2008-DelegatingComputation.pdf

本書での役割

GKR プロトコルの原点。Virgo を理解するための基本文献。

後続研究・注意喚起

[VirgoRevisit25]

Changchang Ding, Yan Huang,
Revisiting Virgo: A Study of Vulnerabilities, Limitations, and Optimizations,
Cybersecurity, Volume 8, Article 118, 2025.
DOI: `10.1186/s42400-025-00450-w`
- DOI: https://doi.org/10.1186/s42400-025-00450-w - Springer: https://link.springer.com/article/10.1186/s42400-025-00450-w

本書での役割

Virgo の zkVPD 設計・実装に関する脆弱性、フィールド制約、range check 負担、改良版提案についての注意喚起。

[FSFRI23]

Alexander R. Block, Albert Garreta, Jonathan Katz, Justin Thaler, Pratyush Ranjan Tiwari, Michał Zając,
Fiat-Shamir Security of FRI and Related SNARKs,
Advances in Cryptology –ASIACRYPT 2023, pp. 3–40, 2023.
DOI: `10.1007/978-981-99-8724-5_1`
- DOI: https://doi.org/10.1007/978-981-99-8724-5_1 - ePrint: https://eprint.iacr.org/2023/1071

本書での役割

FRI 系プロトコルの Fiat–Shamir 後の安全性を精密に考えるための後続研究。

周辺文献（比較や発展のために）

[Libra19]

Tiancheng Xie, Jiaheng Zhang, Yupeng Zhang, Charalampos Papamanthou, Dawn Song,
Libra: Succinct Zero-Knowledge Proofs with Optimal Prover Computation,
CRYPTO 2019, pp. 733–764, 2019.
DOI: `10.1007/978-3-030-26954-8_24`
- DOI: https://doi.org/10.1007/978-3-030-26954-8_24 - ePrint: https://eprint.iacr.org/2019/317

読むとよい理由

GKR 系・多項式評価系の文脈を広げるのに有益。

[Spartan20]

Srinath Setty,
Spartan: Efficient and General-purpose zkSNARKs without Trusted Setup,
CRYPTO 2020, pp. 704–737, 2020.
DOI: `10.1007/978-3-030-56877-1_25`
- DOI: https://doi.org/10.1007/978-3-030-56877-1_25 - ePrint: https://eprint.iacr.org/2019/550

読むとよい理由

transparent / general-purpose / no trusted setup という比較軸で Aurora・Virgo を相対化しやすい。

[Fractal20]

Alessandro Chiesa, Dev Ojha, Nicholas Spooner,
Fractal: Post-Quantum and Transparent Recursive Proofs from Holography,
EUROCRYPT 2020, pp. 769–793, 2020.
DOI: `10.1007/978-3-030-45721-1_27`
- DOI: https://doi.org/10.1007/978-3-030-45721-1_27 - ePrint: https://eprint.iacr.org/2019/1076

読むとよい理由

Aurora 周辺の透明・再帰的 proof の流れを見るのに有益。

読み進め方の提案

もし本書を読み終えたあとに文献へ進むなら、次の順番がおすすめです。

1. [Aurora19] と [Virgo20] の abstract と introduction を読み直す
2. [IOP16] と [FRI18] で Aurora 側の基礎を固める
3. [GKR08] で Virgo 側の原型を読む
4. [VirgoRevisit25] で「原論文をそのまま実装してよいとは限らない」ことを確認する
5. [FSFRI23] で Fiat–Shamir 後の安全性の見方を広げる

最後に

Aurora と Virgo は、どちらも「透明な zk 証明系」という表面上のラベル以上の意味を持っています。

- Aurora は **R1CS を低次数コード語の世界へ運ぶ**という設計思想
- Virgo は **layered circuit を GKR と評価委任で圧縮する**という設計思想

を体現しています。

この 2 本を押さえると、現代の zk 証明の多くが、どの系譜に立っているかをかなり整理して見られるようになります。